

1. Summary

Fully Homomorphic Encryption (FHE) provides a powerful framework for preserving the privacy of sensitive data by enabling computations directly on encrypted data. However, developing efficient FHE programs presents significant challenges, including noise management, vectorization, and performance optimization. Equality saturation emerges as a promising technique to address these challenges by enabling systematic and efficient vectorization of FHE code. This study explores state-of-art FHE compliers and current applications of equality saturation.

Key words : Fully Homomorphic Encryption, Equality saturation, Automatic code optimization, Vectorization

2. Context

In recent years, the growing reliance on cloud-based data processing has raised serious concerns about the privacy of sensitive information, such as health and financial data. Fully Homomorphic Encryption (FHE) offers a strong cryptographic solution by enabling computations to be performed directly on encrypted data. While FHE provides strong security guarantees, it remains computationally demanding, making it an active area for research.

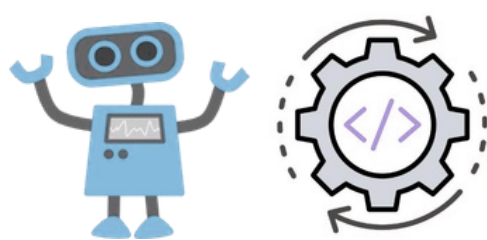
3. Problematic

Performance Challenges in FHE: FHE suffers from high computational overhead, due to expensive operations, and the need for noise management.

Limitations of Traditional Optimization Techniques: Traditional compiler optimizations are ineffective for FHE programs because they don't account for FHE-specific constraints like noise growth and expensive operations, leading to inefficient code.

Adapting Equality Saturation for FHE: To optimize FHE code effectively, equality saturation must be adapted to FHE's unique semantics and cost model, enabling better exploration of equivalent, less costly program rewrites.

How to use equality saturation for FHE code optimization ?



4. Objectives

- Study primitives and building blocks of FHE schemes.
- Study equality saturation and its current applications.
- Review and synthesize recent research on FHE compilers and equality saturation-based optimization frameworks.

5. Fully Homomorphic encryption

Definition :

- FHE allows computations directly on encrypted data without decryption.
- Ensures privacy in untrusted environments (e.g., cloud computing).

Key Properties :

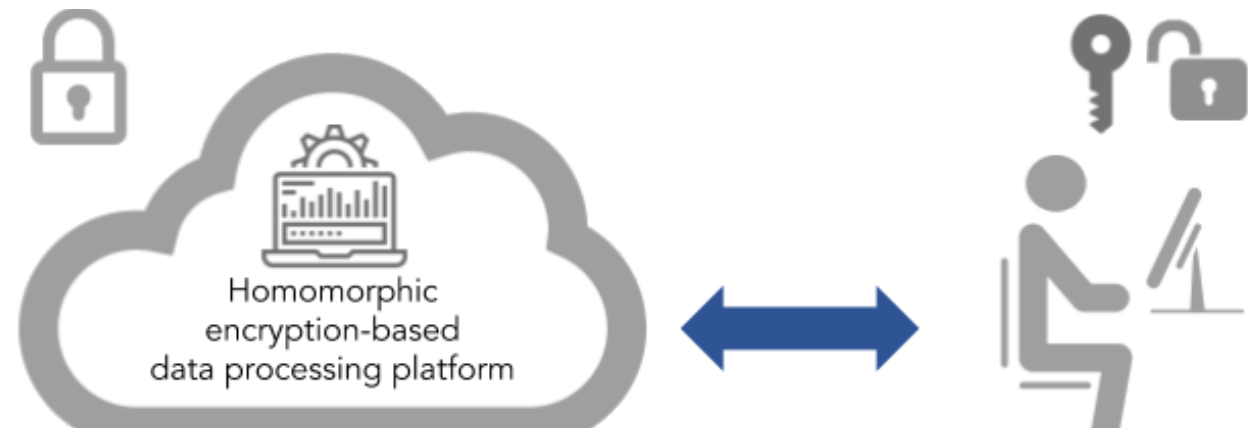
- Supports arbitrary addition and multiplication on ciphertexts.
- Use rotations to access individual slots of used vectors.

Popular Schemes :

- BFV: Integer arithmetic, suitable for exact computations.
- CKKS: Approximate arithmetic, ideal for real numbers.
- TFHE: Bit-level operations, supports fast bootstrapping.

Main Challenges :

- Noise Accumulation: Each operation adds noise, risking data corruption.
- Bootstrapping: Refreshes ciphertext to reduce noise, but is costly.
- Vectorization (SIMD): Packs multiple values into one ciphertext, boosts efficiency but complicates data layout and requires expensive rotations.



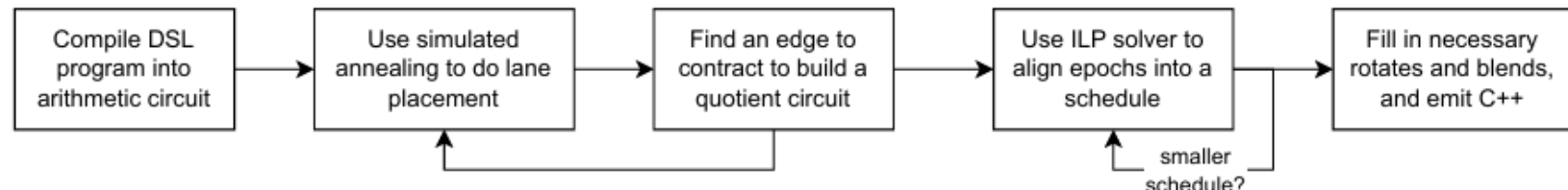
6. FHE Compilers

HECO [1]

- End-to-end optimizing compiler.
- Built on top of MLIR framework.
- Transforms high-level imperative programs into efficient FHE implementations.

Coyote [2]

- introduces an optimized vectorization strategy tailored for FHE.
- Avoids over-vectorization by selectively packing instructions to reduce data movement overhead.
- Exploits Iterative techniques such as simulated annealing and best-first search.

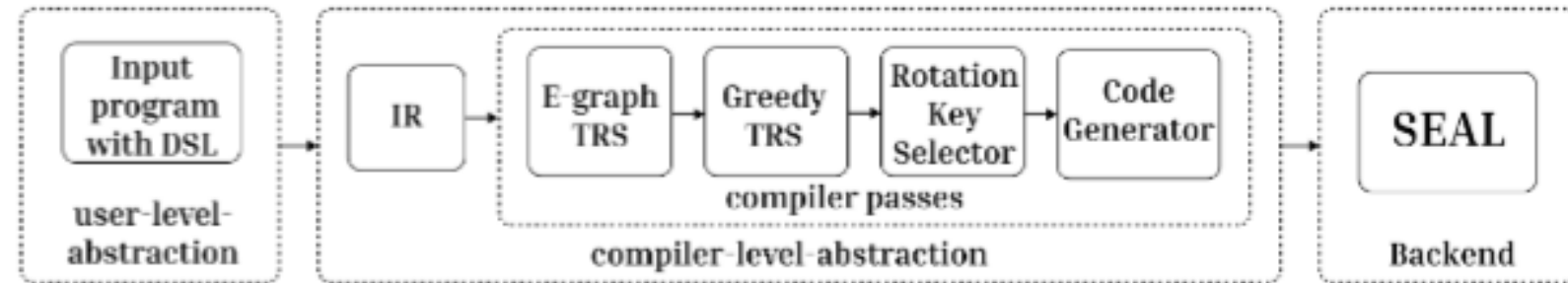


Porcupine [3]

- A synthesizing compiler that automatically generates efficient FHE kernels from plaintext reference implementations.
- Captures FHE-specific constraints, through its intermediate representation called Quill.
- It ensures correctness using symbolic execution and SMT solvers.

CHEHAB

- An FHE compiler that automatically optimizes code by minimizing noise growth and circuit latency.
- Introduces a domain-specific language (DSL) embedded in C++ to abstract FHE concepts.
- It performs two optimization passes:
 - The first pass uses an equality saturation-based term rewriting system (TRS) to perform vectorization.
 - The second pass applies a greedy TRS to reduce noise growth by simplifying arithmetic expressions.



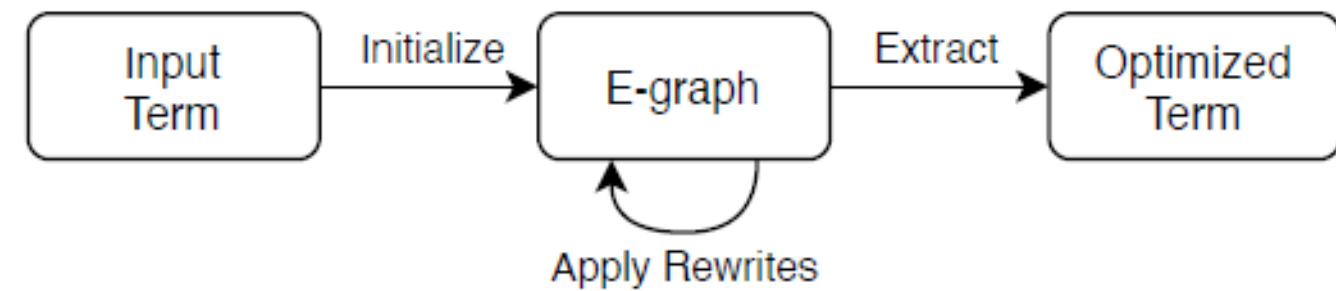
7. Equality saturation

Definition :

- e-graph is a data structure that compactly stores all semantically equivalent forms of a program.
- Instead of applying rewrites sequentially, it accumulates all possible rewrites in parallel, preserving program variants.

Benefits :

- Avoids the phase ordering problem.
- Ensures more global optimization by exploring a larger space of transformations.



8. Equality saturation and Compilers

Diospyros [4]

- A search-based compiler designed to generate efficient vectorized code for Digital Signal Processors (DSPs).
- Uses equality saturation to systematically explore all equivalent program transformations.

Caviar [5]

- An e-graph-based TRS for optimizing and simplifying algebraic expressions in compilers.
- Leverages equality saturation to systematically explore all equivalent forms of an expression.
- This approach ensures comprehensive and effective optimization.

Spores [6]

- An optimizing compiler focused on simplifying linear algebra (LA) expressions.
- It transforms LA expressions into relational algebra (RA) to facilitate reasoning and simplification.
- Uses e-graphs and equality saturation to systematically explore all equivalent rewrites.

9. Synthesis

Compiler	Coyote	Heeo	Porcupine	CHEHAB
SIMD Batching	✓	✓	✓	✓
Reducing Depth	✗	✓	✓	✓
Automatic Parameter Selection	✗	✓	✗	✓
Open Source	✓	✓	✗	✗
Scheme	BFV	BFV	BFV, CKKS	BFV
Prog. Language	Python	Python	Racket	C++

10. Limitations and perspectives

Scalability: Equality saturation can lead to large e-graphs, making it memory and time-intensive, especially for complex FHE circuits.

Cost Models: Developing FHE-aware cost models is essential for selecting efficient rewrites, especially for operations like rotations and multiplications.

Hybrid Approaches: Combining equality saturation with greedy or heuristic methods could balance exploration and performance.

11. Bibliographic References

[1] Viand, Alexander et al. (2023). "HECO: Fully Homomorphic Encryption Compiler." In: Proceedings of the 32nd USENIX Security Symposium (cit. on pp. 1, 34). [2] Malik, Raghav et al. (2023). "Coyote: A Compiler for Vectorizing Encrypted Arithmetic Circuits." In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 118–133 (cit. on pp. 1, 33). [3] Cowan, Meghan et al. (2021). "Porcupine: A Synthesizing Compiler for Vectorized Homomorphic Encryption." In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), pp. 375–389 (cit. on pp. 1, 6, 35) . [4] VanHattum, Alexa et al. (2021). "Vectorization for digital signal processors via equality saturation." In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 874–886 (cit. on p. 32). [5] Kourta, Smail et al. (2022). "Caviar: an e-graph based TRS for automatic code optimization." In: Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, pp. 54–64 (cit. on p. 31). [6] Wang, Yisu Remy et al. (2020). "SPORES: sum-product optimization via relational equality saturation for large scale linear algebra." In: arXiv preprint arXiv:2002.07951 (cit. on p. 31) .

1. Summary

Fully Homomorphic Encryption (FHE) provides a powerful framework for preserving the privacy of sensitive data by enabling computations directly on encrypted data. However, developing efficient FHE programs presents significant challenges, including noise management, vectorization, and performance optimization. Equality saturation emerges as a promising technique to address these challenges by enabling systematic and efficient vectorization of FHE code. This study explores state-of-art FHE compliers and current applications of Equality saturation.

Key words : Fully Homomorphic Encryption, Equality saturation, Automatic code optimization, Vectorization

2. Context

In recent years, the growing reliance on cloud-based data processing has raised serious concerns about the privacy of sensitive information, such as health and financial data. Fully Homomorphic Encryption (FHE) offers a strong cryptographic solution by enabling computations to be performed directly on encrypted data. While FHE provides strong security guarantees, it remains computationally demanding, making its an active area for practical deployment.

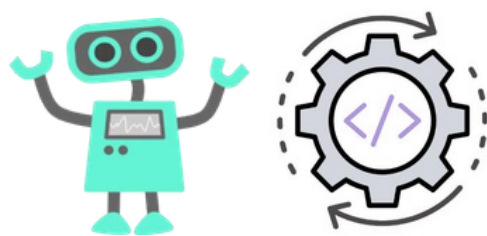
3. Problematic

Performance Challenges in FHE: FHE suffers from high computational overhead, due to expensive operations like ciphertext rotations, multiplications, and the need for noise management.

Limitations of Traditional Optimization Techniques: Traditional compiler optimizations are not designed with the constraints of FHE in mind, such as noise growth, and the high cost of operations. As a result, they produce inefficient code when applied to FHE programs.

Adapting Equality Saturation for FHE: Equality saturation, a powerful automatic code optimization technique , must be tailored to the semantics and cost model of FHE in order to effectively explore equivalent program rewrites and reduce expensive operations.

How to use equality saturation for FHE code optimization ?



4. Objectives

- Study primitives and building blocks of FHE schemes.
- Study equality saturation and its current applications.
- Review and synthesize recent research on FHE compilers and equality saturation-based optimization frameworks.

5. Fully Homomorphic encryption

Definition :

- FHE allows computations directly on encrypted data without decryption.
- Ensures privacy in untrusted environments (e.g., cloud computing).

Key Properties :

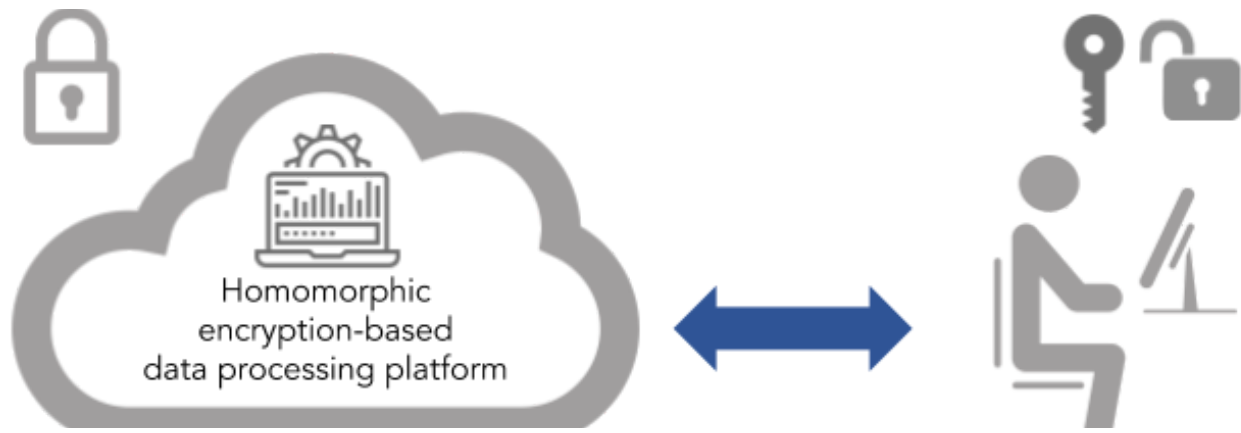
- Supports arbitrary addition and multiplication on ciphertexts.
- Use rotations to access individual slots of used vectors.

Popular Schemes :

- BFV: Integer arithmetic, suitable for exact computations.
- CKKS: Approximate arithmetic, ideal for real numbers.
- TFHE: Bit-level operations, supports fast bootstrapping.

Main Challenges :

- Noise Accumulation: Each operation adds noise, risking data corruption.
- Bootstrapping: Refreshes ciphertext to reduce noise, but is costly.
- Vectorization (SIMD): Packs multiple values into one ciphertext, boosts efficiency but complicates data layout and requires expensive rotations.



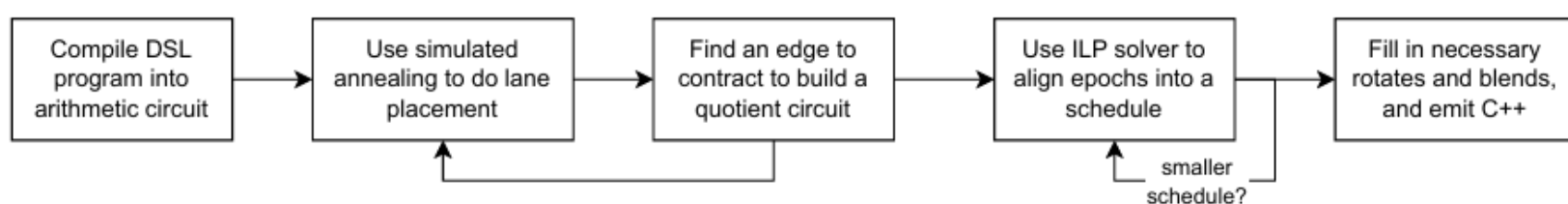
6. FHE Compilers

HECO [1]

HECO is an end-to-end optimizing compiler Built on top of MLIR framework. It transforms high-level imperative programs into efficient and secure implementations compatible with FHE.

Coyote [2]

Coyote proposes an optimized vectorization strategy for FHE that avoids over-vectorization by selectively packing instructions to limit data movement overhead. Using iterative techniques like simulated annealing and best-first search, Coyote generate high quality vectorized code that takes into account the constraints of FHE.

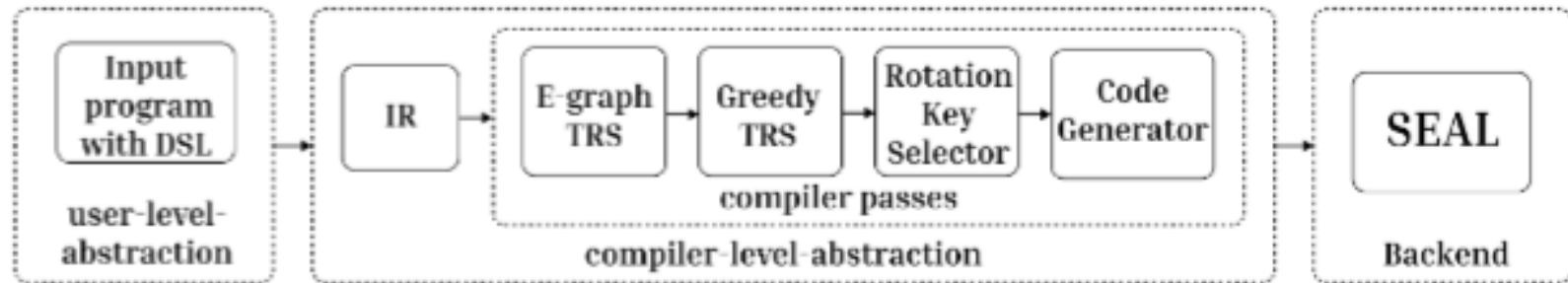


Porcupine [3]

Porcupine is a synthesizing compiler that automates the creation of efficient FHE kernels from a plaintext reference implementation. It captures FHE-specific constraints, such as noise growth and latencythrough its intermediate representation, Quill, and ensures correctness using symbolic execution and SMT solvers.

CHEHAB

CHEHAB is a compiler for FHE that automatically optimizes code by minimizing noise growth and circuit latency. It introduces a DSL embedded in C++ to abstract FHE concepts and allow developer to write high-level code without managing FHE operations. The compilation pipeline begins by lowering the DSL into an intermediate representation (IR), followed by two optimization passes: the first uses an equality saturation-based term rewriting system (TRS) to perform vectorization, and the second applies a greedy TRS to reduce noise growth by simplifying arithmetic expressions.



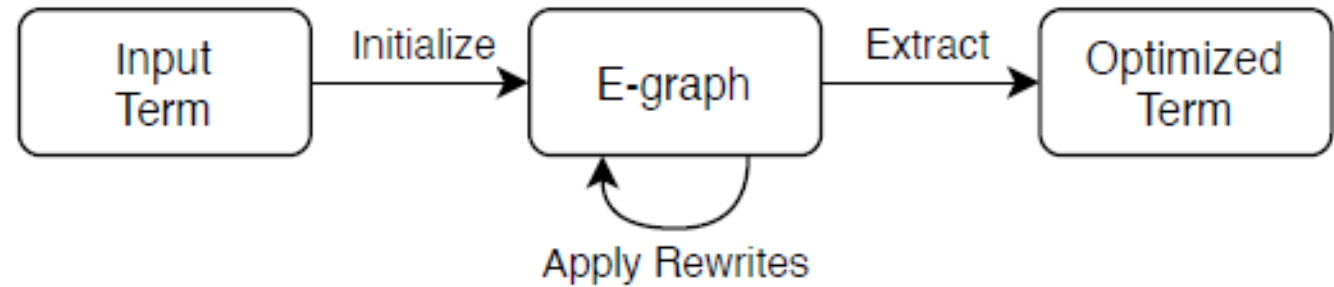
7. Equality saturation

Definition :

- e-graph is a data structure that compactly stores all semantically equivalent forms of a program.
- Instead of applying rewrites sequentially, it accumulates all possible rewrites in parallel, preserving program variants.

Benefits :

- Avoids the phase ordering problem.
- Ensures more global optimization by exploring a larger space of transformations.



8. Equality saturation and Compilers

Diospyros [4]

Diospyros is a search-based compiler designed to generate efficient vectorized code for Digital Signal Processors. It uses equality saturation to systematically explore all equivalent program transformations, enabling the discovery of highly efficient vectorization strategies.

Caviar [5]

Caviar is an e-graph-based TRS designed to optimize and simplify algebraic expressions in compilers. It leverages equality saturation to systematically explore all possible equivalent forms of an expression, ensuring comprehensive optimization.

Spores [6]

SPORES is an optimizing compiler designed to simplify linear algebra (LA) expressions. It transforms LA into relational algebra (RA), making it easier to reason about and simplify. By leveraging e-graphs and equality saturation, SPORES systematically explores all equivalent rewrites and uses a constraint solver to select the most efficient form based on a cost model.

9. Synthesis

Compiler	Coyote	HECO	Porcupine	CHEHAB
SIMD Batching	✓	✗	✗	✗
Reducing Depth	✓	✓	✓	✗
Automatic Parameter Selection	✓	✓	✗	✗
Open Source	✓	✓	✓	✗

10. Limitations and perspectives

Scalability: Equality saturation can lead to large e-graphs, making it memory and time-intensive, especially for complex FHE circuits.

Cost Models: Developing FHE-aware cost models is essential for selecting truly efficient rewrites, especially for operations like rotations and bootstrapping.

Hybrid Approaches: Combining equality saturation with greedy or heuristic methods could balance exploration and performance.

11. Bibliographic References

[1] Viand, Alexander et al. (2023). “HECO: Fully Homomorphic Encryption Compiler.” In: Proceedings of the 32nd USENIX Security Symposium (cit. on pp. 1, 34). [2] Malik, Raghav et al. (2023). “Coyote: A Compiler for Vectorizing Encrypted Arithmetic Circuits.” In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), pp. 118–133 (cit. on pp. 1, 33). [3] Cowan, Meghan et al. (2021). “Porcupine: A Synthesizing Compiler for Vectorized Homomorphic Encryption.” In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), pp. 375–389 (cit. on pp. 1, 6, 35) . [4] VanHattum, Alexa et al. (2021). “Vectorization for digital signal processors via equality saturation.” In: Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 874–886 (cit. on p. 32). [5] Kourta, Smail et al. (2022). “Caviar: an e-graph based TRS for automatic code optimization.” In: Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, pp. 54–64 (cit. on p. 31). [6] Wang, Yisu Remy et al. (2020). “SPORES: sum-product optimization via relational equality saturation for large scale linear algebra.” In: arXiv preprint arXiv:2002.07951 (cit. on p. 31) .

1. Résumé

L'intégration de l'apprentissage par renforcement dans l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine. **Mots clés** : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

Encadré par : BAGHDADI Riyadh

9. Limitations et perspectives

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLlib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.

Espace de recherche du client RL

- fonctions d'état
- fonctions de récompense
- Actions
- algorithmes de RL
- benchmarks d'entraînement
- Interfaces de mesure

Recherche du client de RL

- Recherche de politique basée sur le bandit multi-bras
- Apprentissage multi-tâche

SUPERSONIC

- L'architecture du client RL choisi
- Ajustement des paramètres du client RL

programmes non vus

- Client RL ajusté
- code optimisé

Déploiement du client RL

re-entraînement et inférence

Définition hors-ligne de la tâche et recherche du client RL : **effectué une fois**

RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.

A blue robot with a screen for a face and a gear with code symbols (</>) inside it, surrounded by red arrows indicating a cycle.

Halide [3]

The diagram illustrates a Reinforcement Learning pipeline for image processing. It consists of three main components in a sequence: 'Pipeline de traitement d'image', 'Ordonnement Génération et Estimation Raffinement', and 'Compilation et exécution'. The first two components are connected by a straight arrow. The last two are connected by a curved arrow labeled 'Temps d'exécution'. A feedback loop is shown with a curved arrow from the end of the 'Compilation et exécution' step back to the 'Ordonnement Génération et Estimation Raffinement' step, labeled 'Option d'ordonnement sélectionné'. Additionally, 'Options d'ordonnement' are provided as input to the 'Ordonnement Génération et Estimation Raffinement' step.

```

graph LR
    A[Programme initial] --> B[Auto-scheduler utilisant l'apprentissage par renforcement]
    B --> C[Séquence d'optimisations]
    C --> D[Programme optimisé]
  
```

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta- optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

Réalisé par : KASMI Abderrahmene & SALMA Samy

Encadré par : BAGHDADI Riyadh

1. Résumé

dans l'intégration de l'apprentissage par renforcement l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine.

Mots clés : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

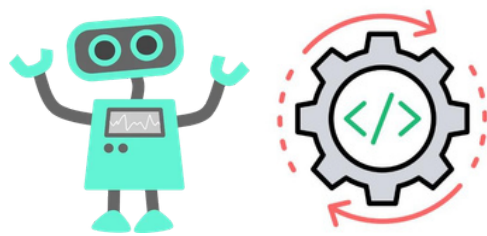
3. Problématique

Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.

Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.

Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

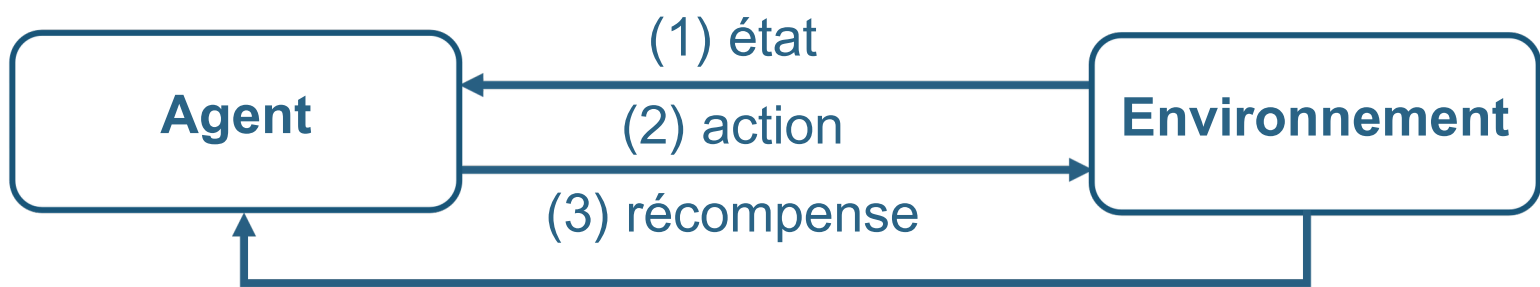
5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



6. Apprentissage par renforcement

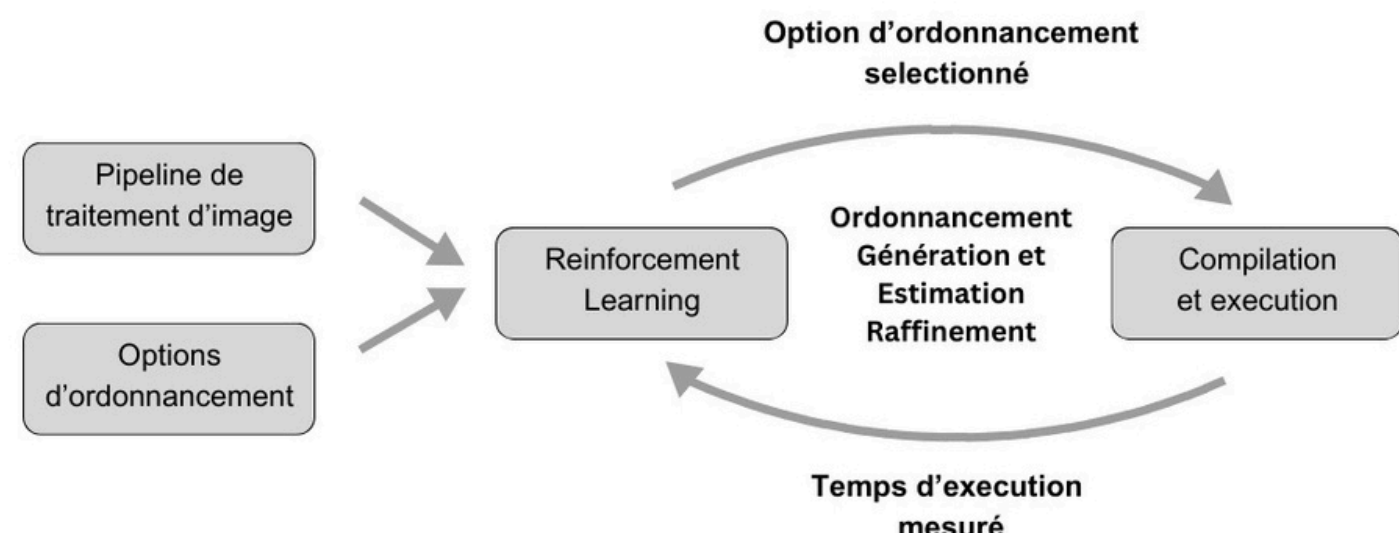
L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



7. Apprentissage par renforcement et compilateurs

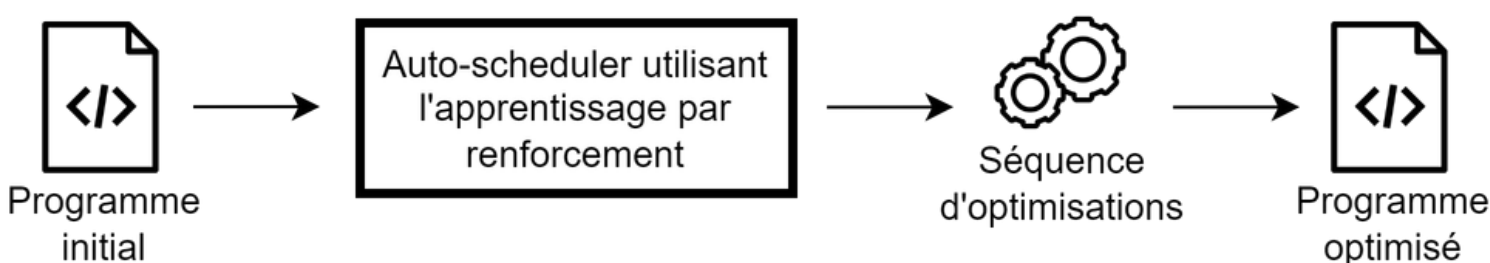
Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



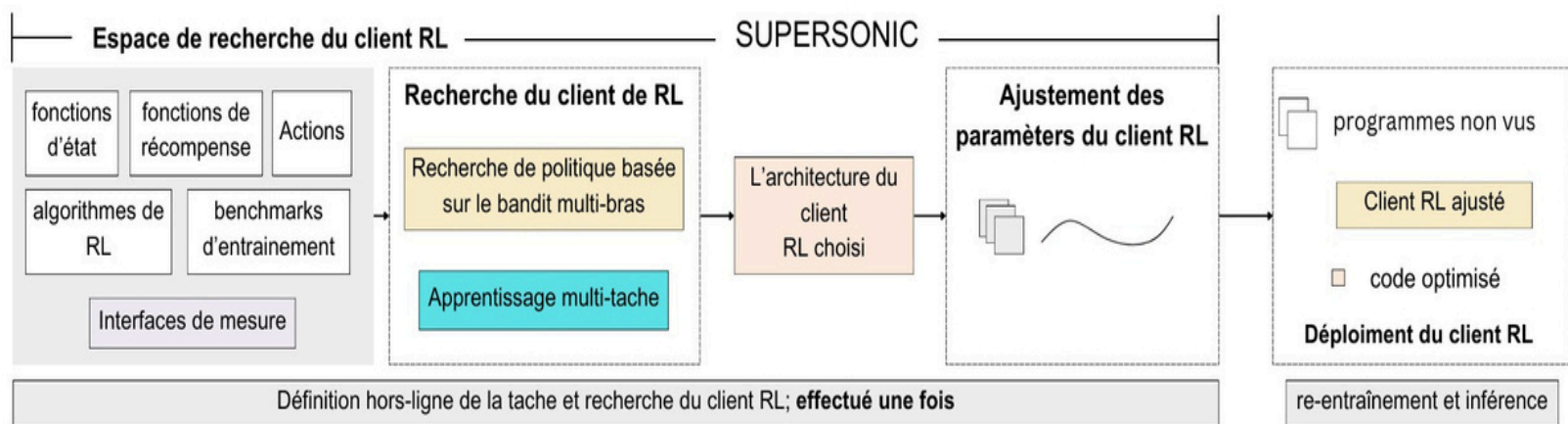
Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



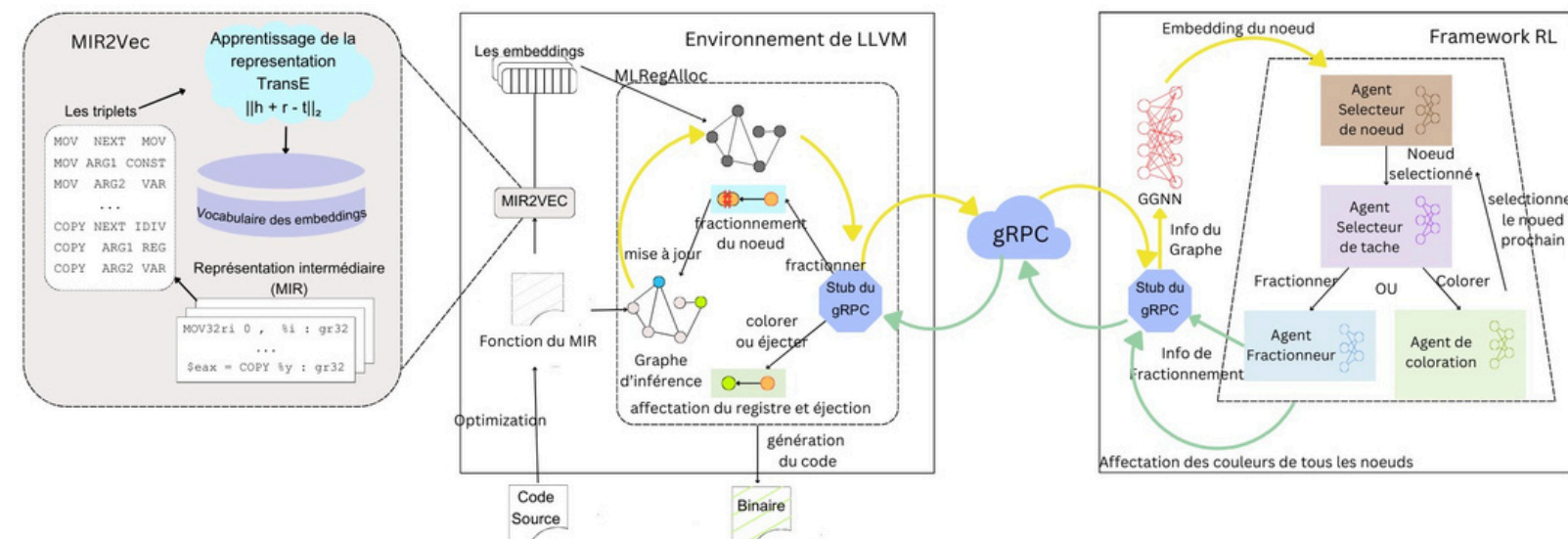
SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement : ajouter des programmes provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

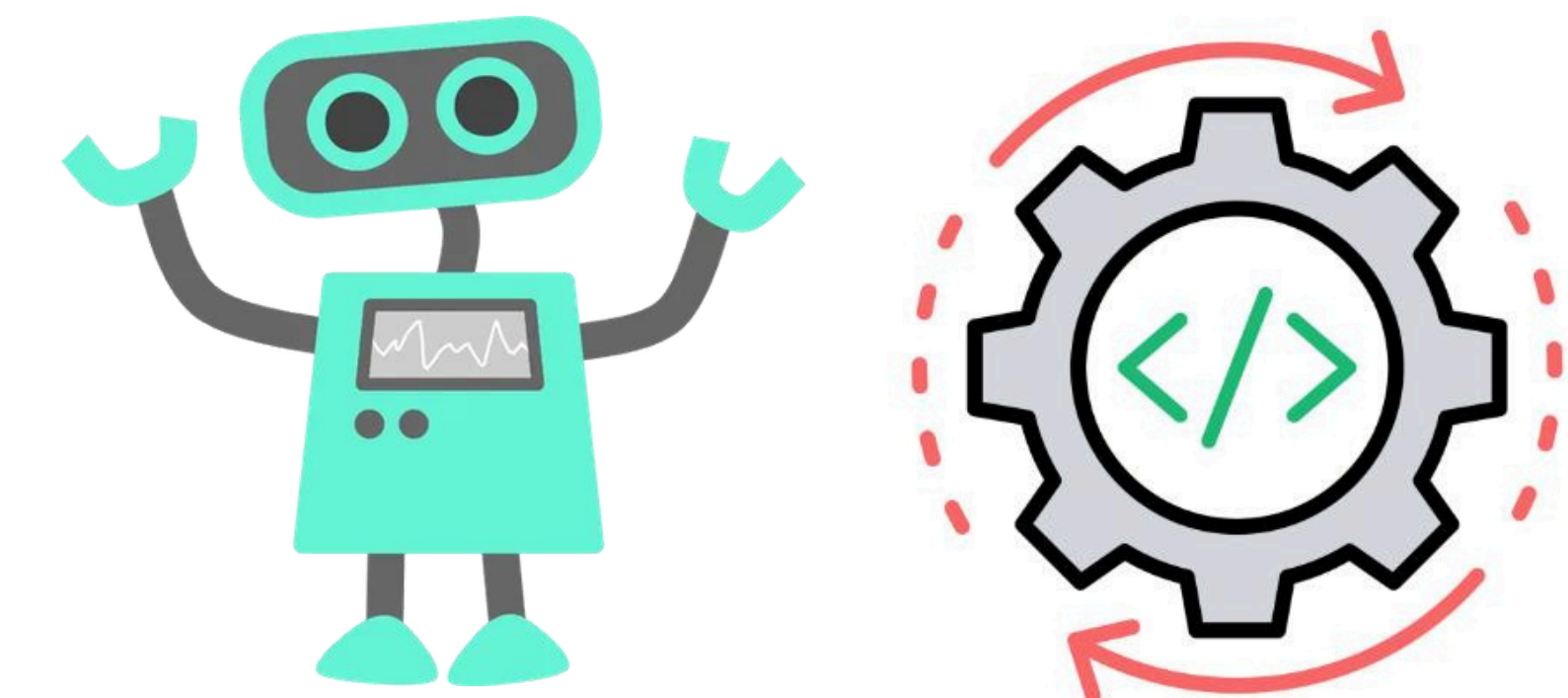
11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas , A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. Acm Sigplan Notices, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAI : Reinforcement Learning for Register Allocation. Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, 129-143.

3. Problématique

- Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.
- Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.
- Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



Encadré par : BAGHDADI Riyadh

9. Limitations et perspectives

permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

3. Problématique

```

graph LR
    A[Programme initial] --> B[Auto-scheduler utilisant l'apprentissage par renforcement]
    B --> C[Séquence d'optimisations]
    C --> D[Programme optimisé]
  
```

The diagram illustrates a four-step process for code optimization using reinforcement learning. It begins with a document icon labeled 'Programme initial'. An arrow points to a box labeled 'Auto-scheduler utilisant l'apprentissage par renforcement'. Another arrow points to a gear icon labeled 'Séquence d'optimisations'. A final arrow points to a document icon labeled 'Programme optimisé'.

[illegible]

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LIVM	LIVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

Encadré par : BAGHDADI Riyadh

9. Limitations et perspectives

permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

8. Synthèse

```

graph LR
    A[Programme initial] --> B[Auto-scheduler utilisant l'apprentissage par renforcement]
    B --> C[Séquence d'optimisations]
    C --> D[Programme optimisé]
  
```

The diagram illustrates the MIR2Vec framework for protein representation learning, divided into several key components and processes:

- Initial Data and Training:**
 - Les triplets:** Input data used for training.
 - Apprentissage de la représentation TransE $[h + r = t_k]$:** The process of learning representations using TransE.
 - Vocabulaire des embeddings:** The resulting vocabulary of embeddings.
 - Représentation intermédiaire (MIR):** The intermediate representation generated from the training process.
 - Optimisation:** The MIR is optimized into **Code Source** (Source Code).
 - Code Source:** The optimized source code, which is then compiled into **Binaires** (Binaries).
- Execution and Embedding Generation:**
 - Environnement de LLVM:** The binaries are executed within the LLVM environment.
 - MIR2Vec:** A component that processes the LLVM environment to generate **Les embeddings** (The embeddings).
- Graph Processing and Communication:**
 - Fractionnement du nœud:** Node partitioning process.
 - mise à jour:** Update process.
 - Graphes d'inférence:** Inference graphs generated from the embeddings.
 - affectation du registre et éjection:** Register assignment and spilling process.
 - gRPC:** A communication protocol used for data exchange between components.
- Reinforcement Learning Framework (Framework RL):**
 - Embedding du nœud:** The node embeddings are used as input.
 - Agent Sélectionneur de tâche:** A task selection agent that chooses between **Fractionner** (fractionate) or **colorer** (color).
 - Agent Fractionneur:** An agent that performs the fractionation process.
 - Agent de coloration:** An agent that performs the coloring process.
 - Nœud sélectionné:** The selected node for processing.
 - sélectionner le répéteur protéin:** Selecting the protein repeat.
 - Affectation des couleurs de tous les nœuds:** Assigning colors to all nodes.

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LIVM	LIVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAl: Reinforcement Learning for Register Allocation. *Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction*, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*, 129-143.

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les
- compilateurs pour améliorer l'optimisation du code.

Synthétiser les approches et les travaux existants dans la littérature.

5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



Réalisé par : KASMI Abderrahmene & SALMA Samy

Encadré par : BAGHDADI Riyadh

1. Résumé

danl'intégration de l'apprentissage par renforcement l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine.

Mots clés : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

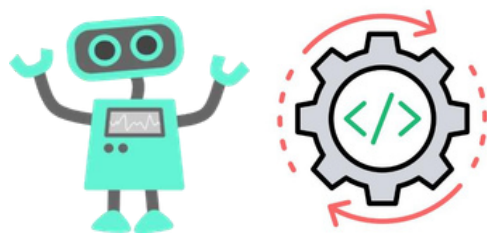
3. Problématique

Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.

Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.

Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

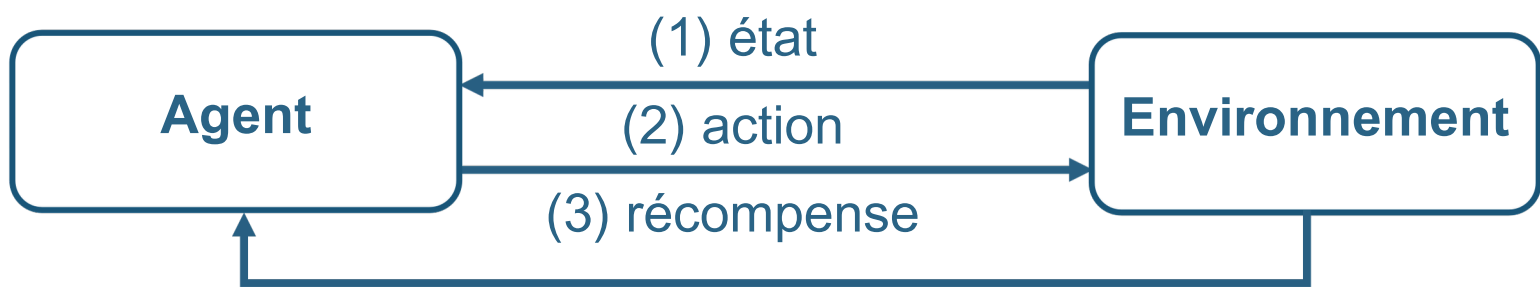
5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



6. Apprentissage par renforcement

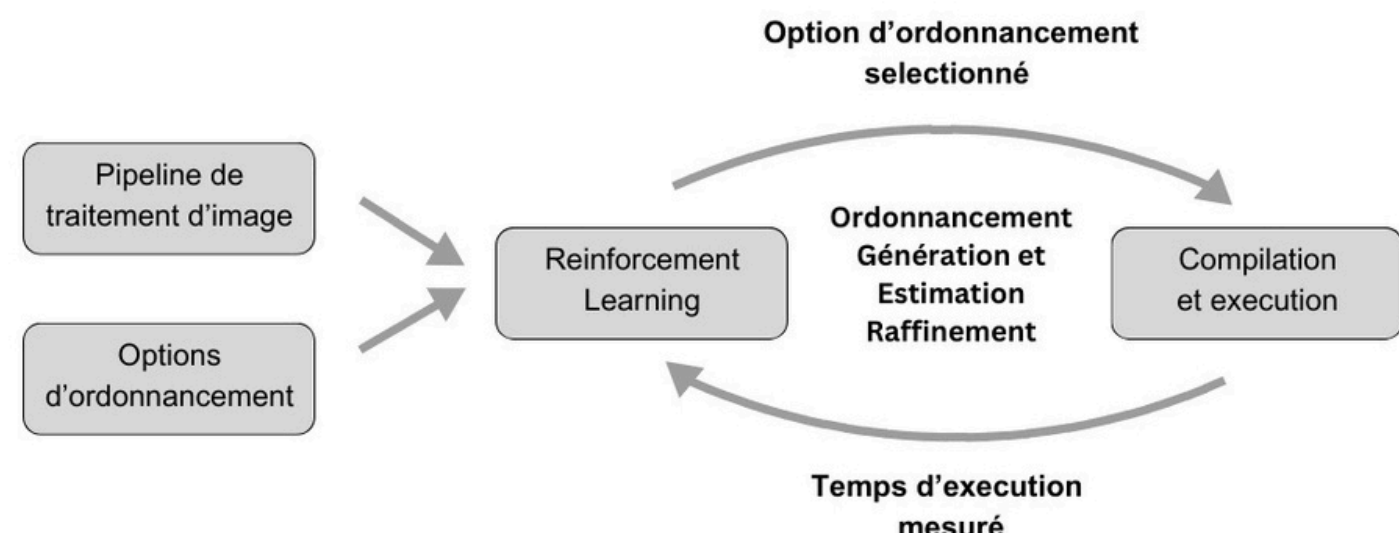
L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



7. Apprentissage par renforcement et compilateurs

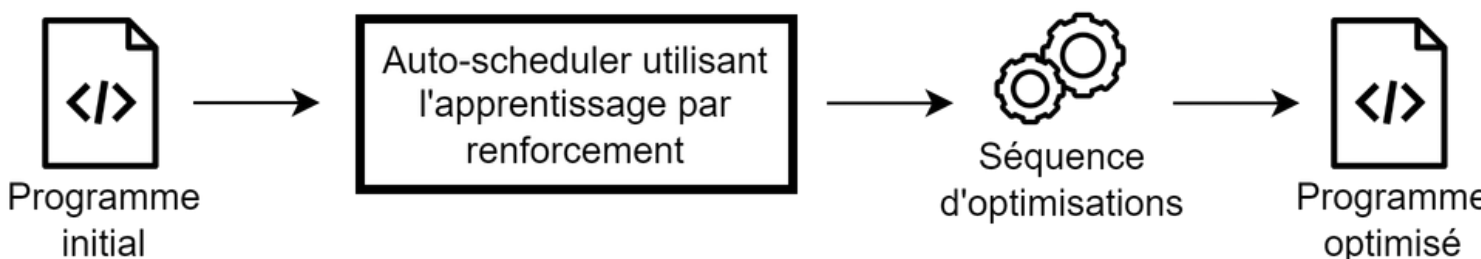
Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



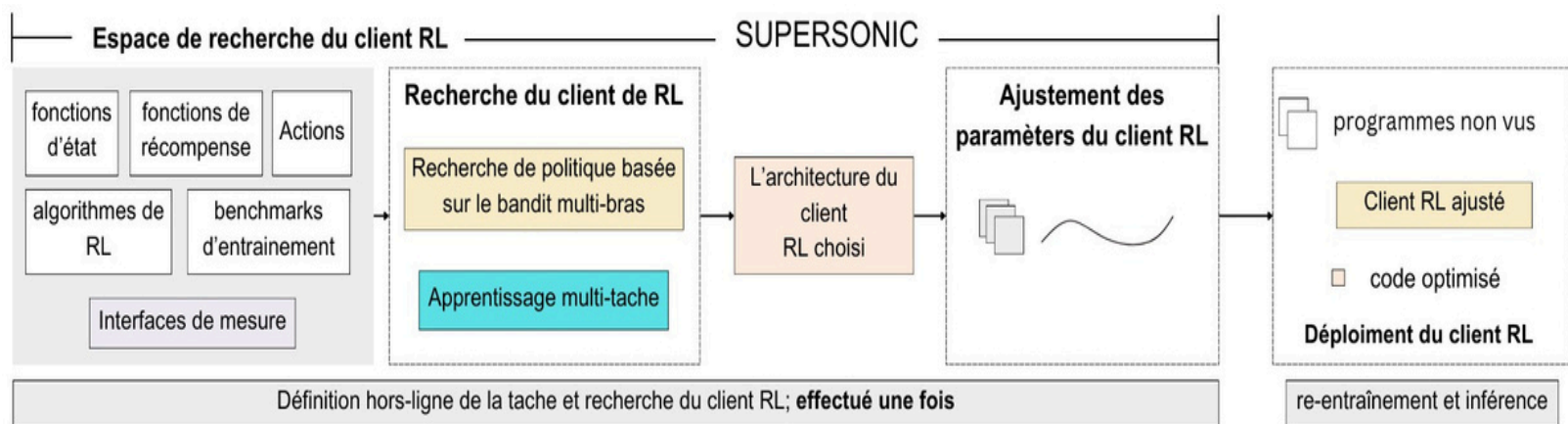
Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



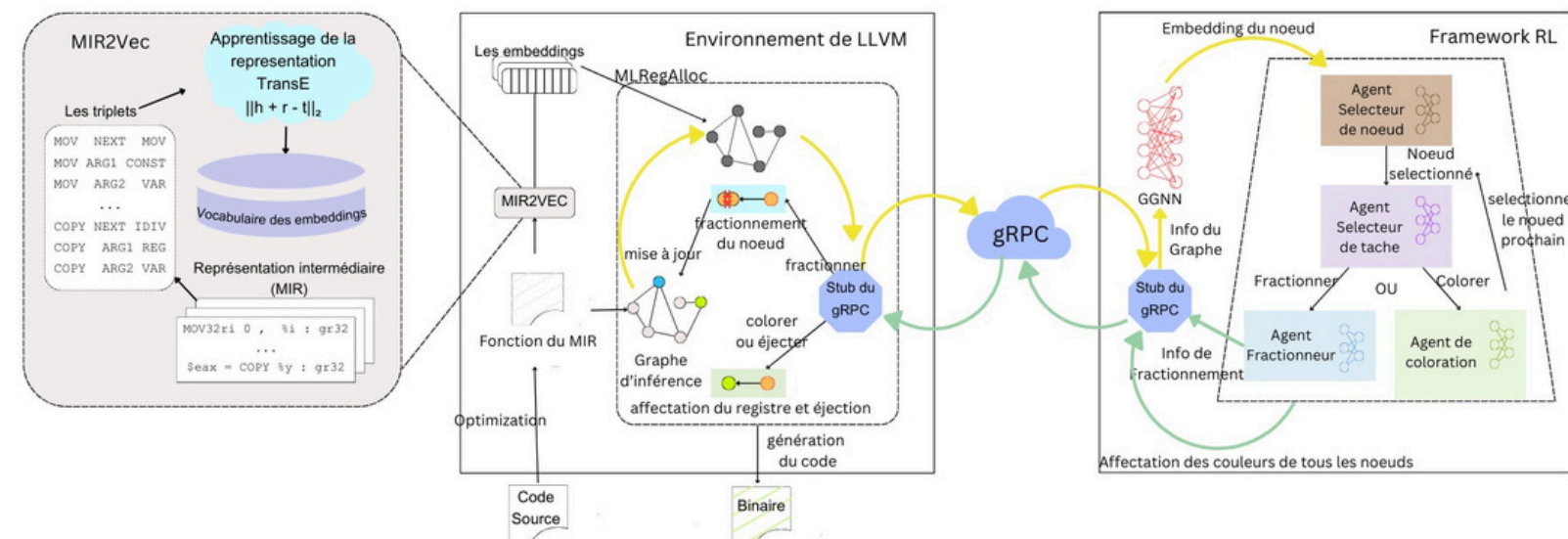
SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement : ajouter des programmes provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

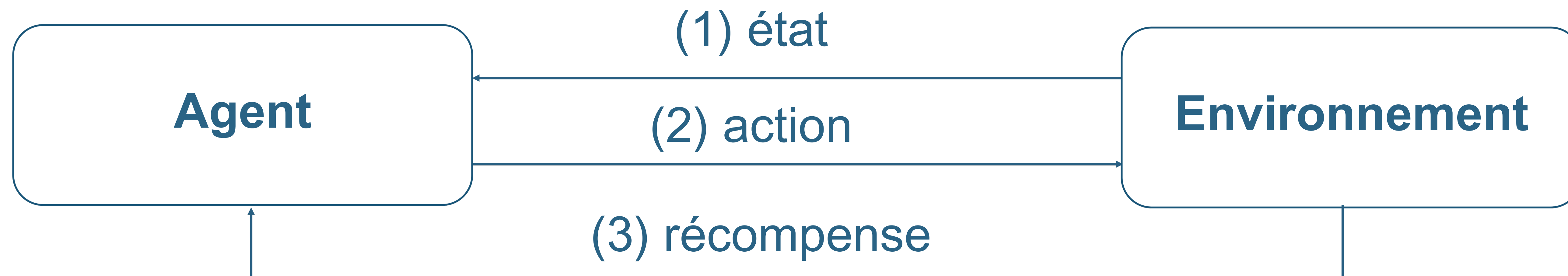
Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas , A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. Acm Sigplan Notices, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAI : Reinforcement Learning for Register Allocation. Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, 129-143.

6. Apprentissage par renforcement

L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



Réalisé par : KASMI Abderrahmene & SALMA Samy

Encadré par : BAGHDADI Riyadh

1. Résumé

danl'intégration de l'apprentissage par renforcement l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine.

Mots clés : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

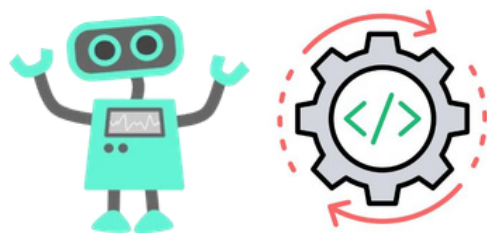
3. Problématique

Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.

Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.

Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

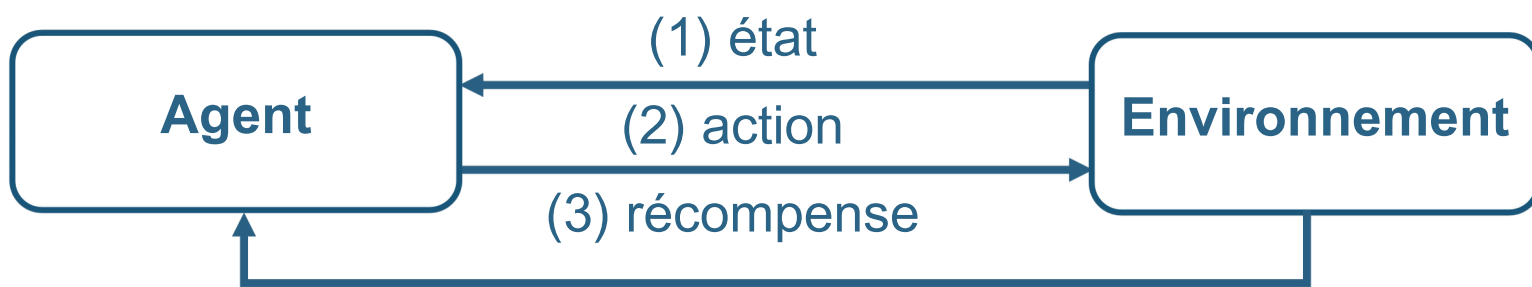
5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



6. Apprentissage par renforcement

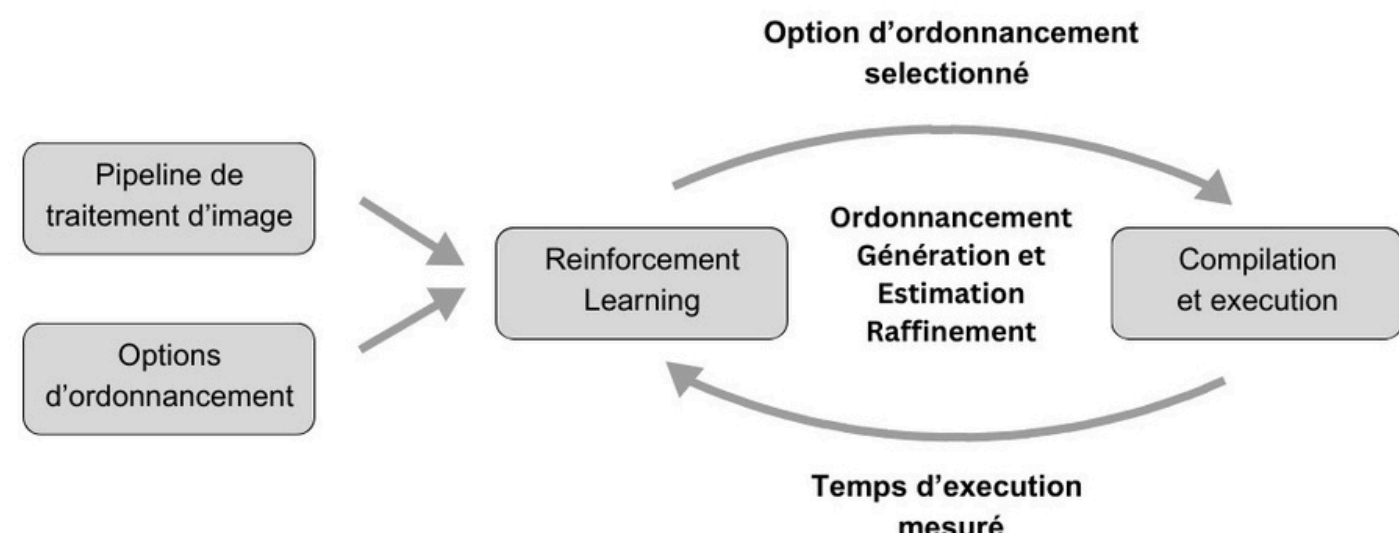
L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



7. Apprentissage par renforcement et compilateurs

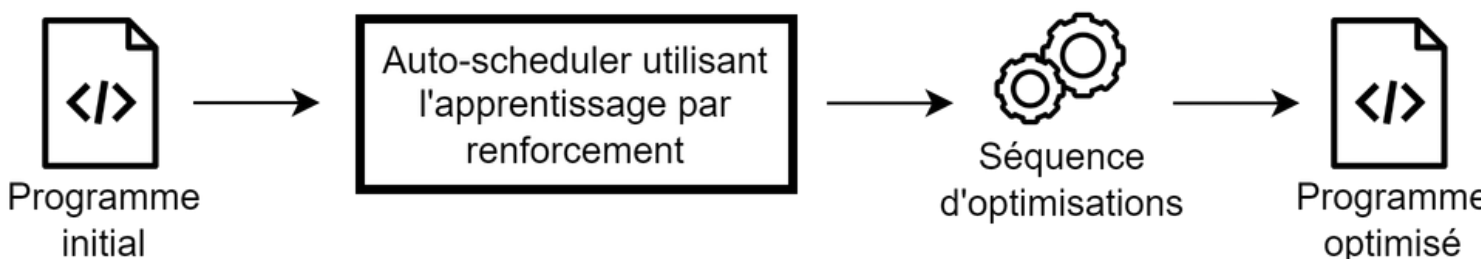
Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



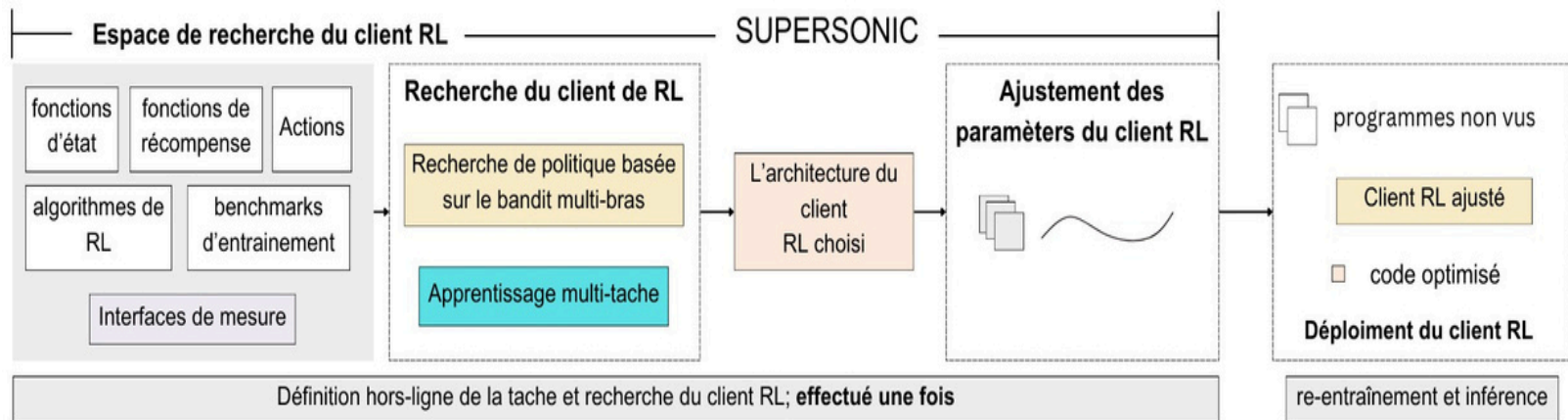
Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



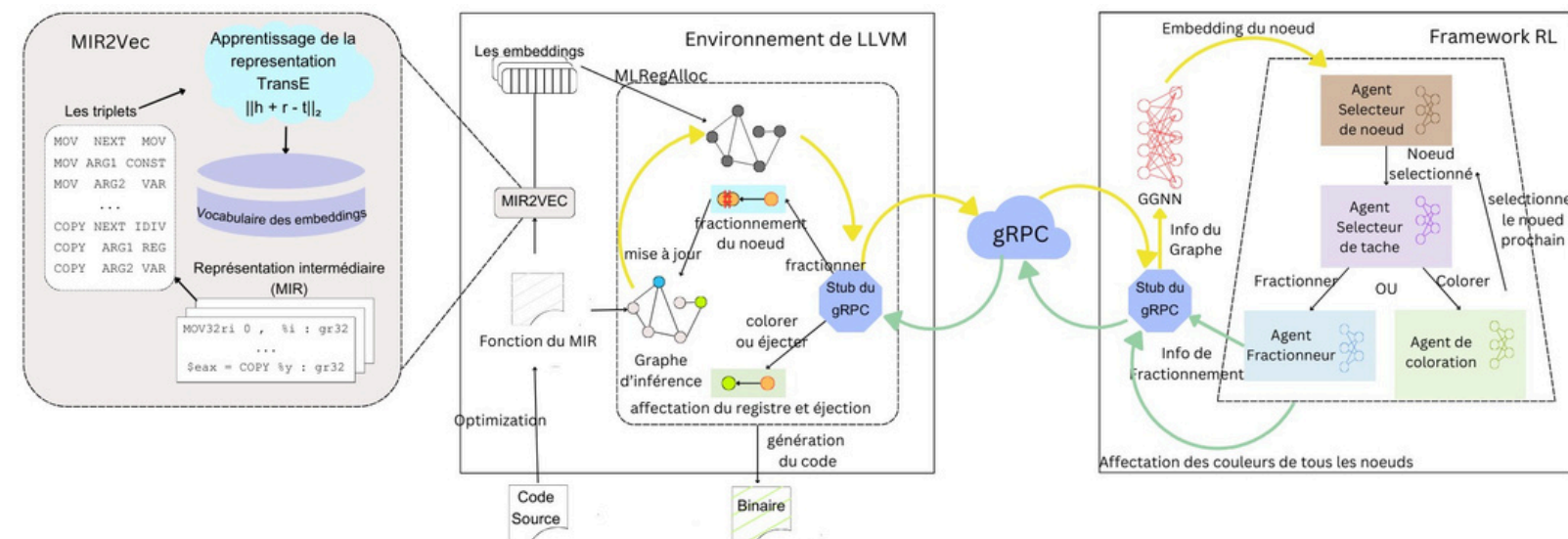
SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement : ajouter des programmes provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

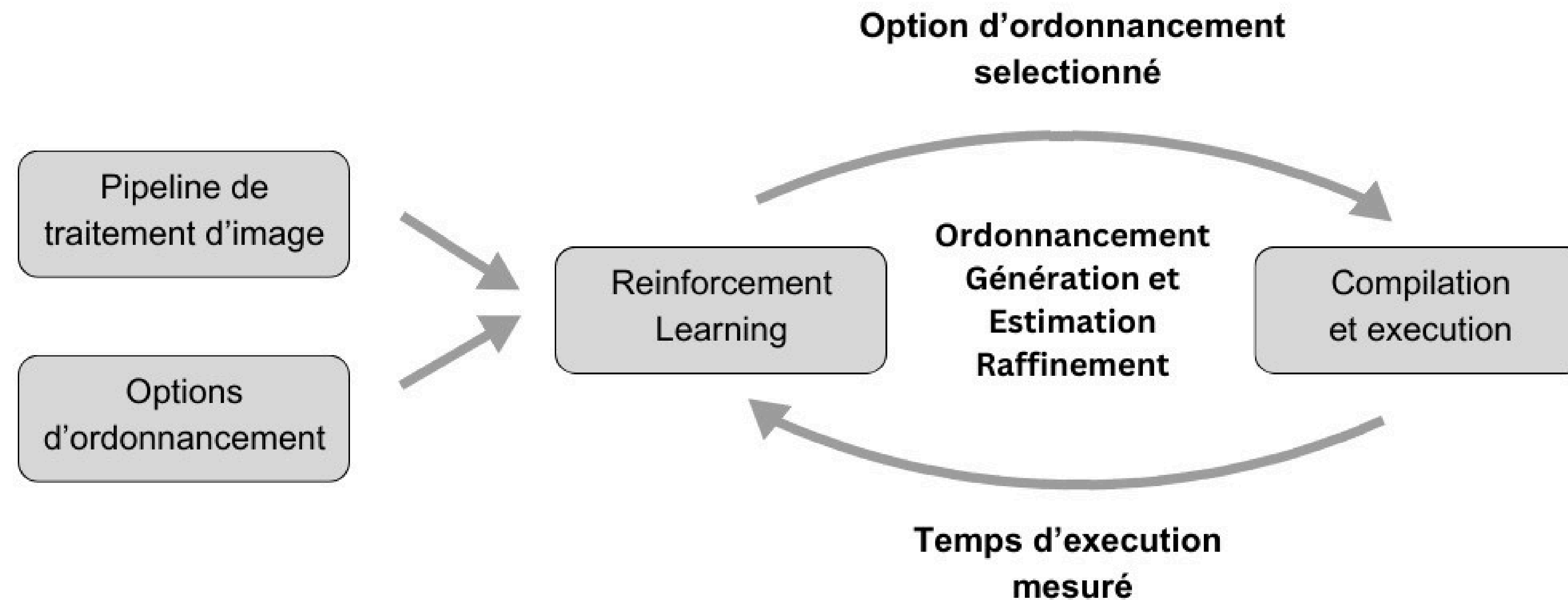
Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas , A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. Acm Sigplan Notices, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAI : Reinforcement Learning for Register Allocation. Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, 129-143.

Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



Réalisé par : KASMI Abderrahmene & SALMA Samy

Encadré par : BAGHDADI Riyadh

1. Résumé

dans l'intégration de l'apprentissage par renforcement l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine.

Mots clés : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

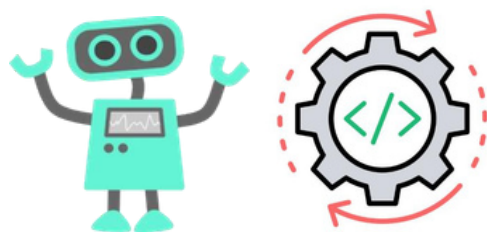
3. Problématique

Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.

Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.

Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

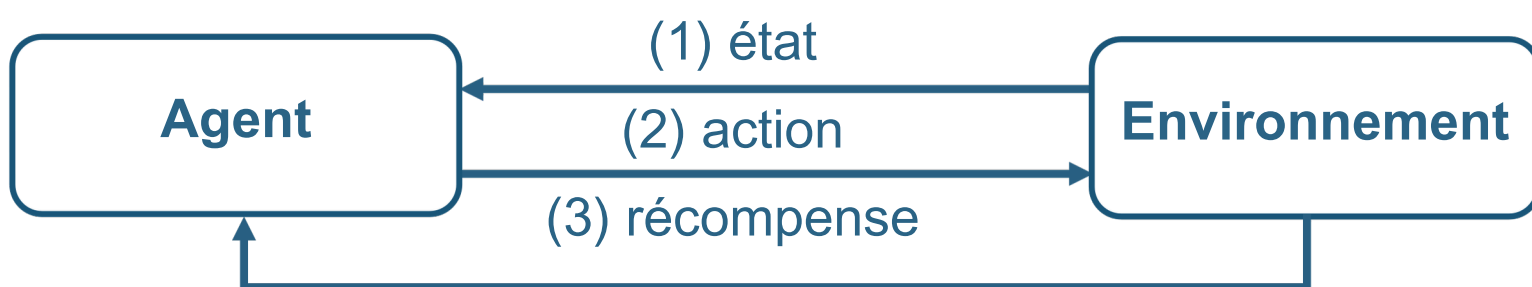
5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



6. Apprentissage par renforcement

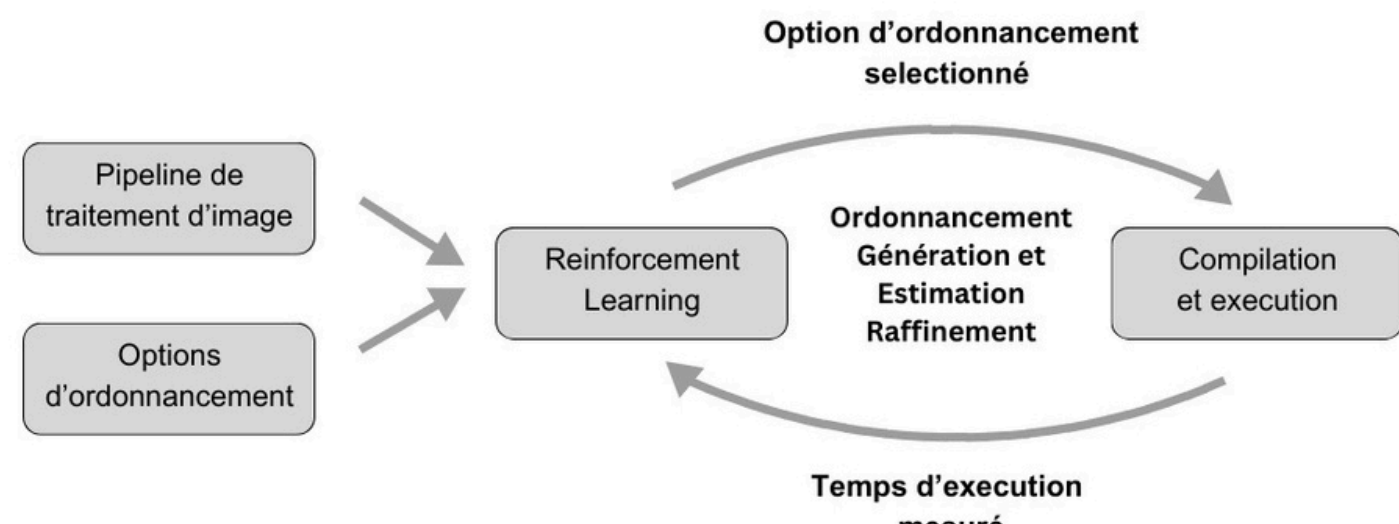
L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



7. Apprentissage par renforcement et compilateurs

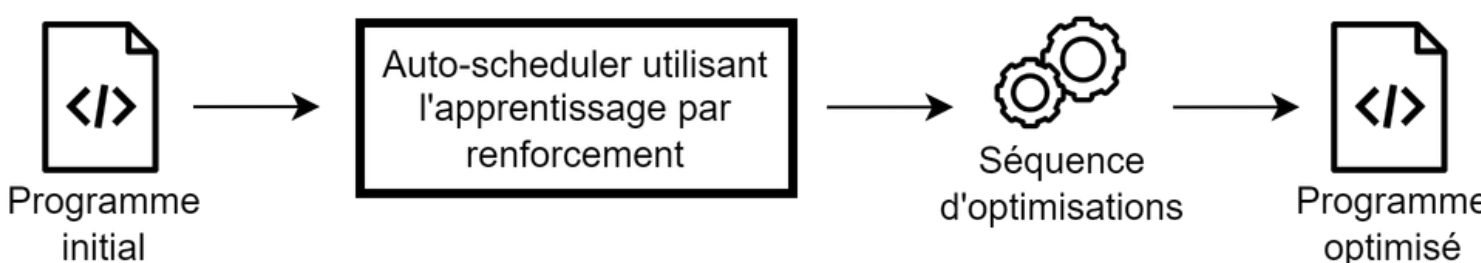
Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



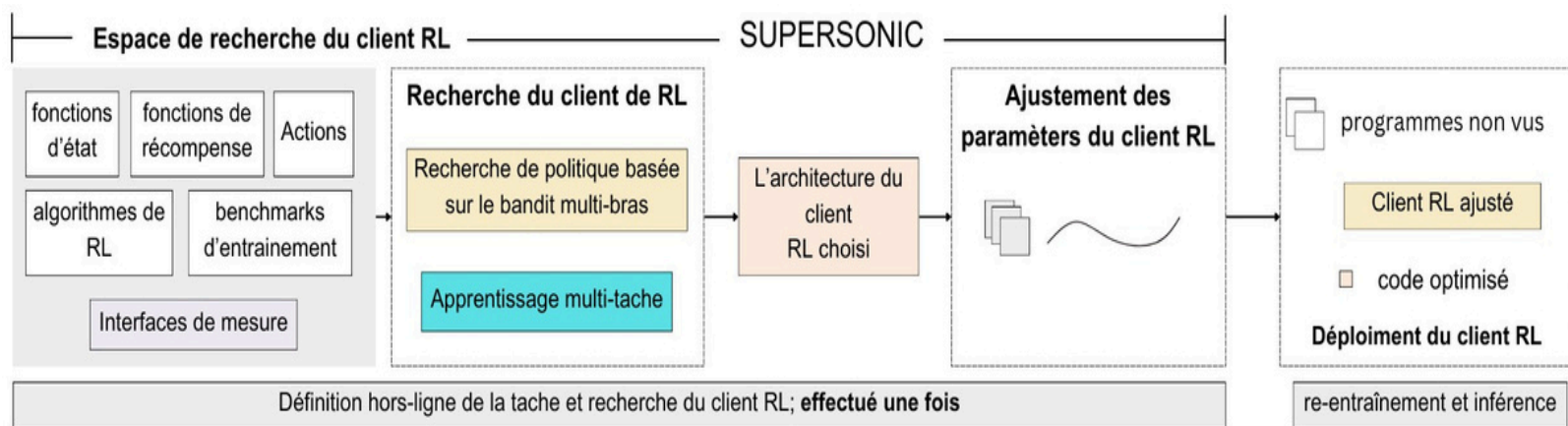
Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



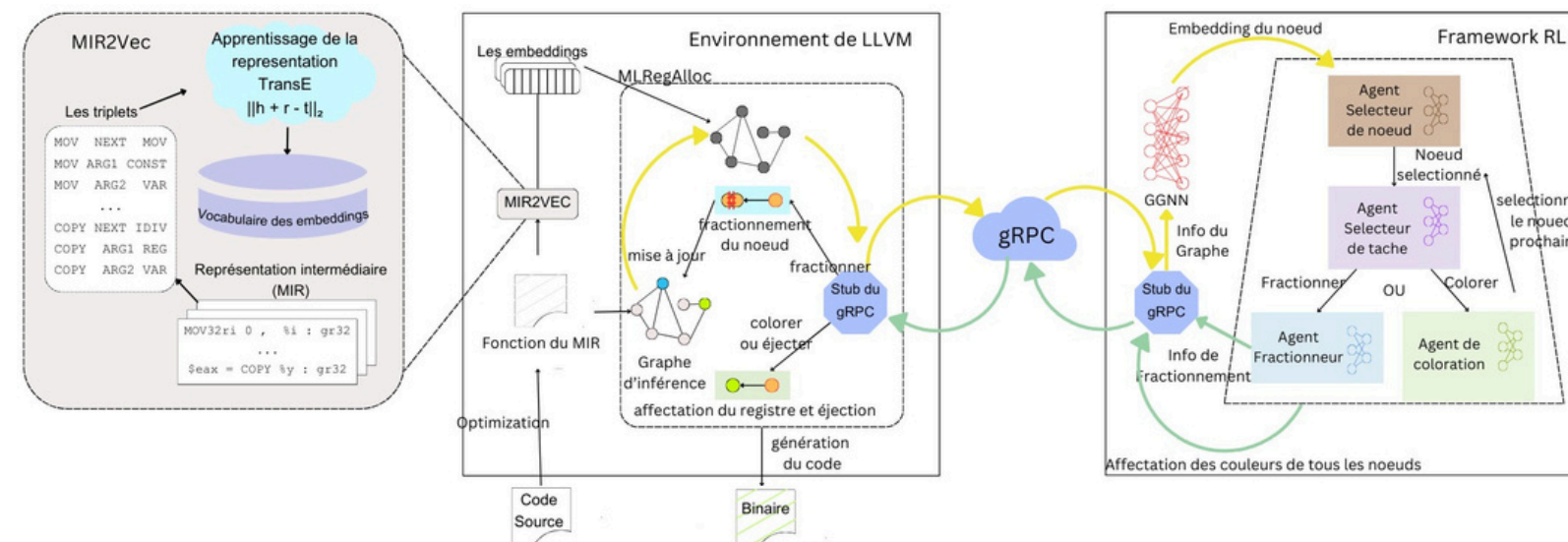
SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement : ajouter des programmes provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

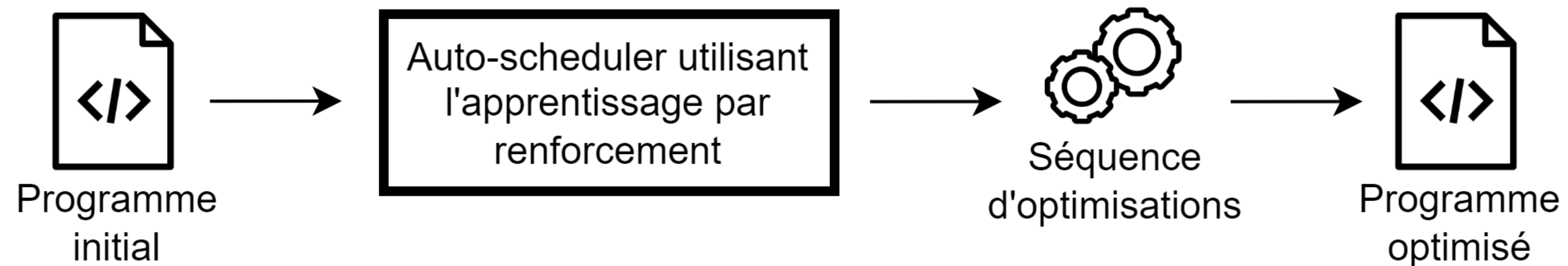
Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas , A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. Acm Sigplan Notices, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAI : Reinforcement Learning for Register Allocation. Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, 129-143.

Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



Réalisé par : KASMI Abderrahmene & SALMA Samy

Encadré par : BAGHDADI Riyadh

1. Résumé

danl'intégration de l'apprentissage par renforcement l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine.

Mots clés : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

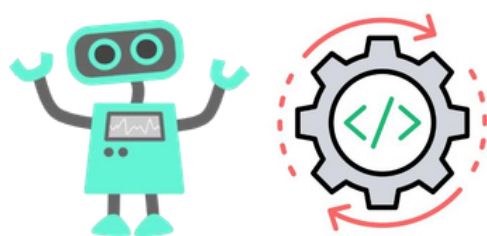
3. Problématique

Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.

Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.

Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

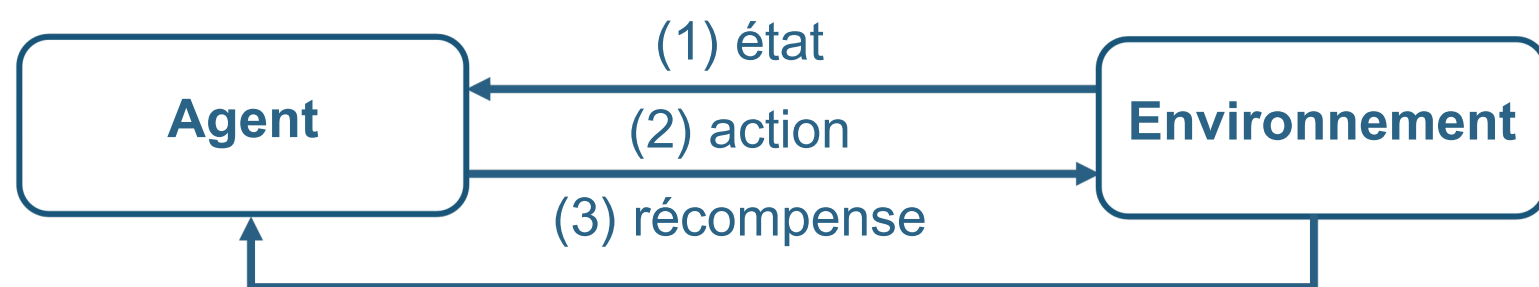
5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



6. Apprentissage par renforcement

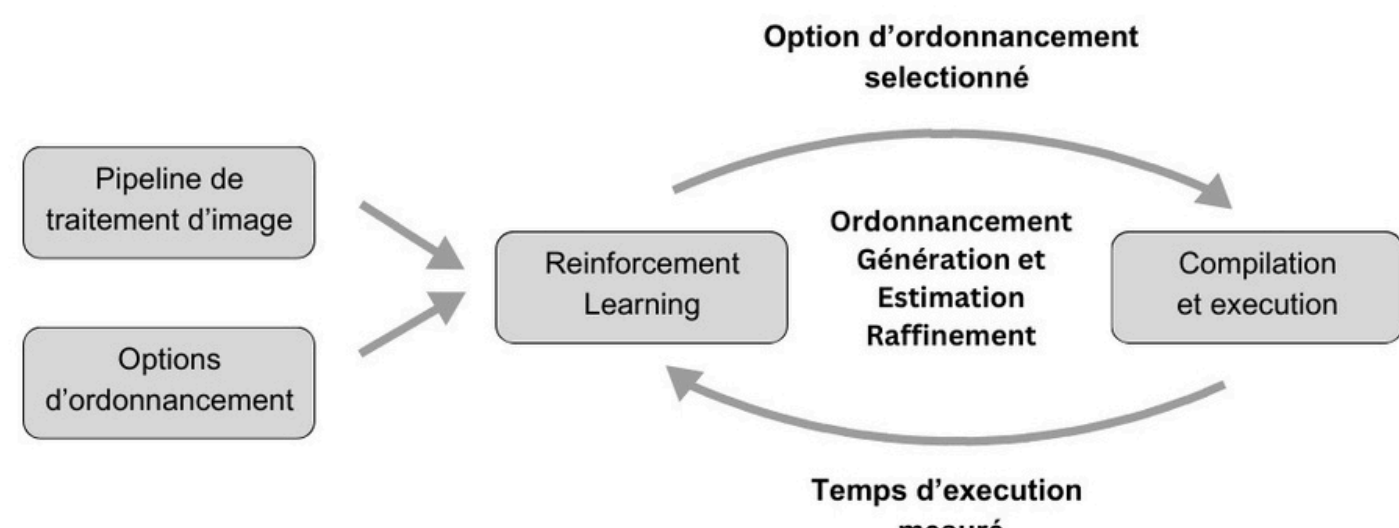
L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



7. Apprentissage par renforcement et compilateurs

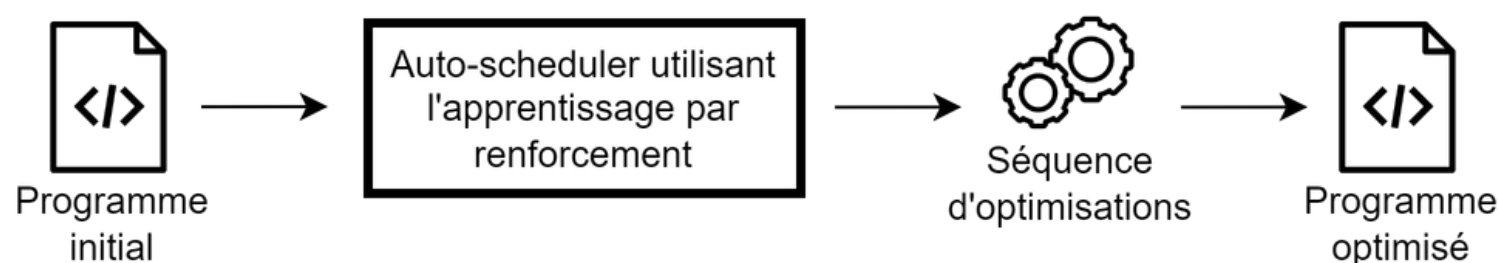
Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



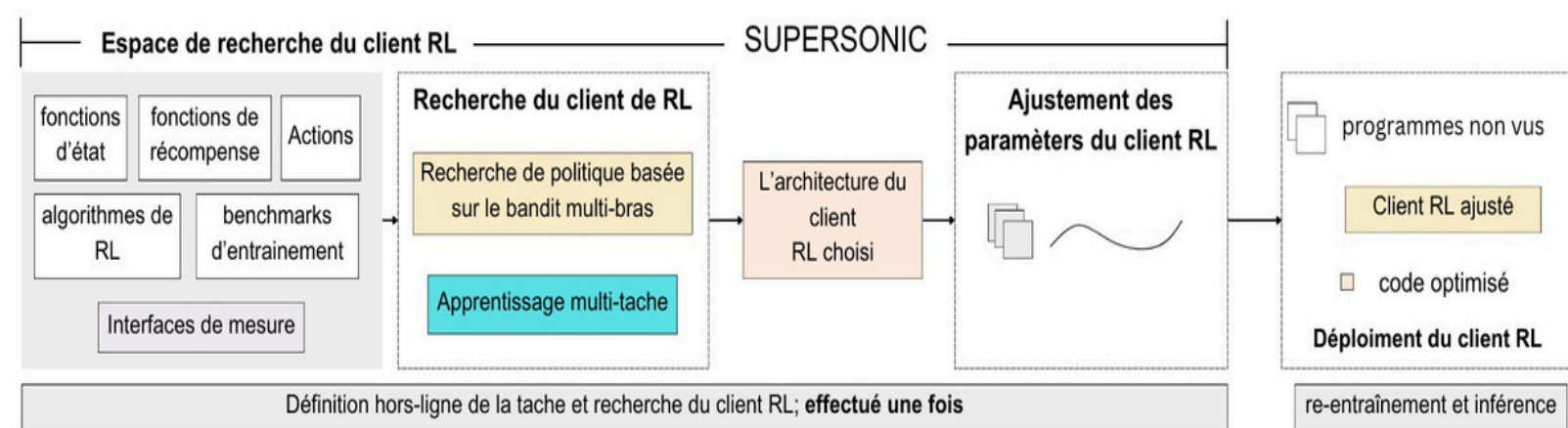
Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



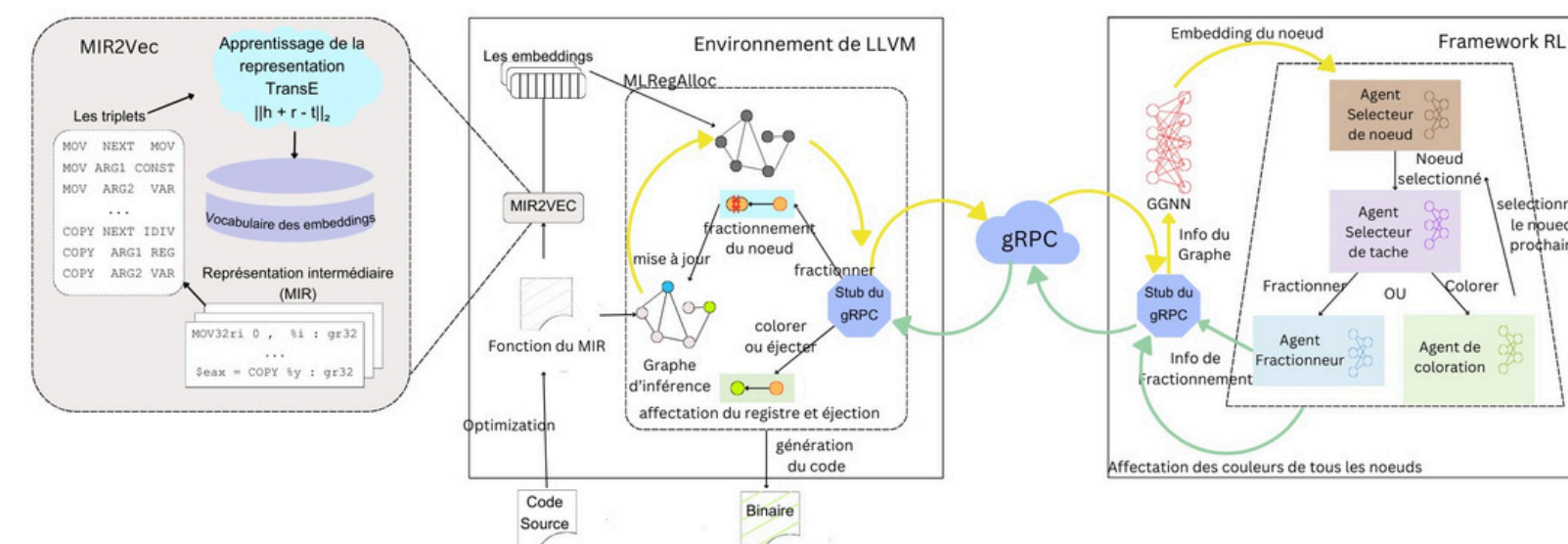
SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement : ajouter des programmes provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

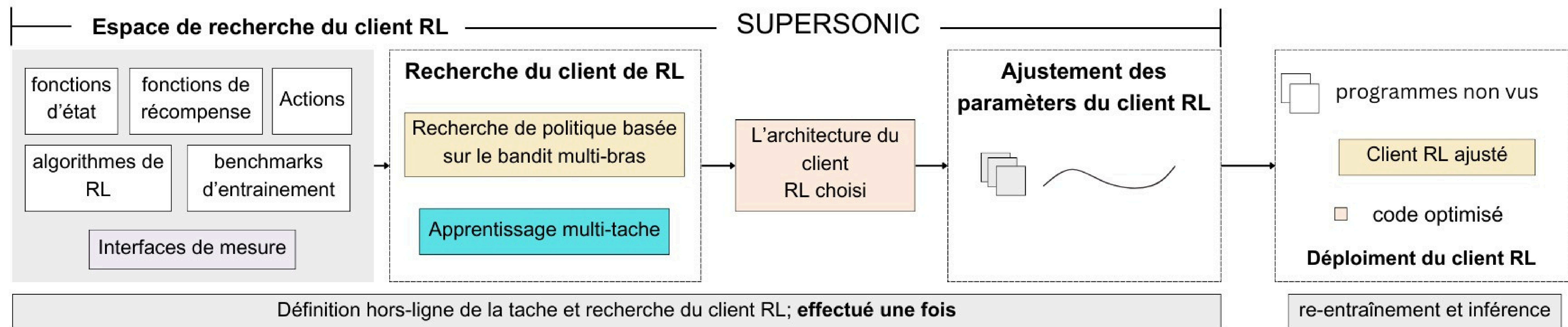
Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas , A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. Acm Sigplan Notices, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAI : Reinforcement Learning for Register Allocation. Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, 129-143.

SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLlib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



Réalisé par : KASMI Abderrahmene & SALMA Samy

Encadré par : BAGHDADI Riyadh

1. Résumé

danl'intégration de l'apprentissage par renforcement l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine.

Mots clés : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

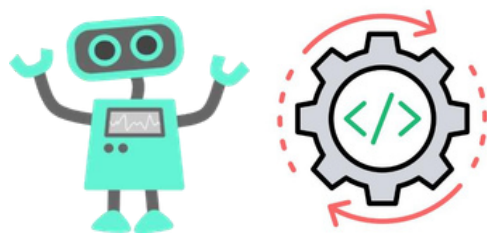
3. Problématique

Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.

Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.

Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

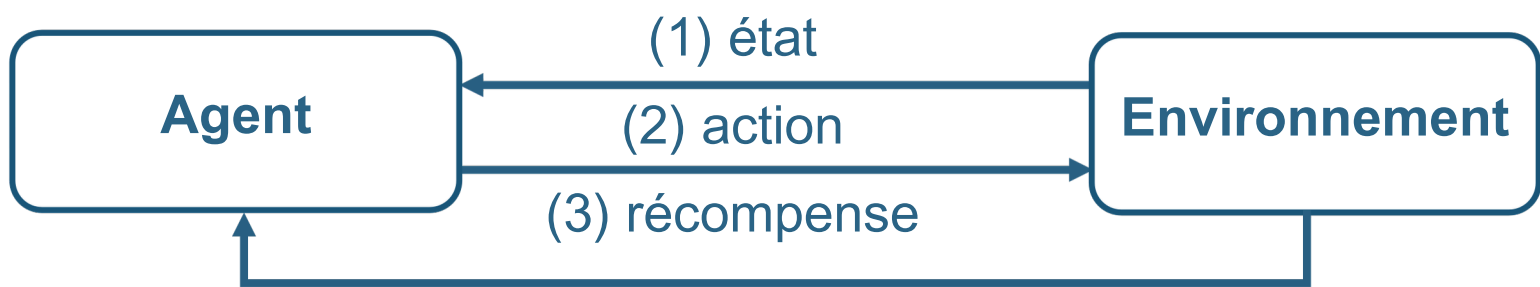
5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



6. Apprentissage par renforcement

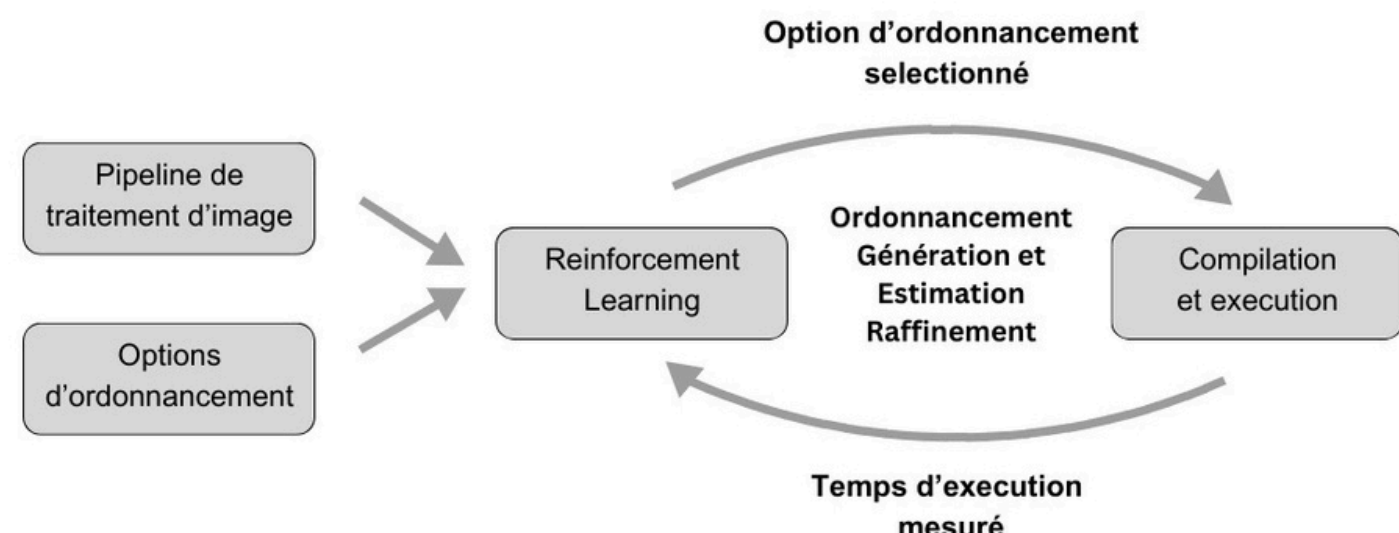
L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



7. Apprentissage par renforcement et compilateurs

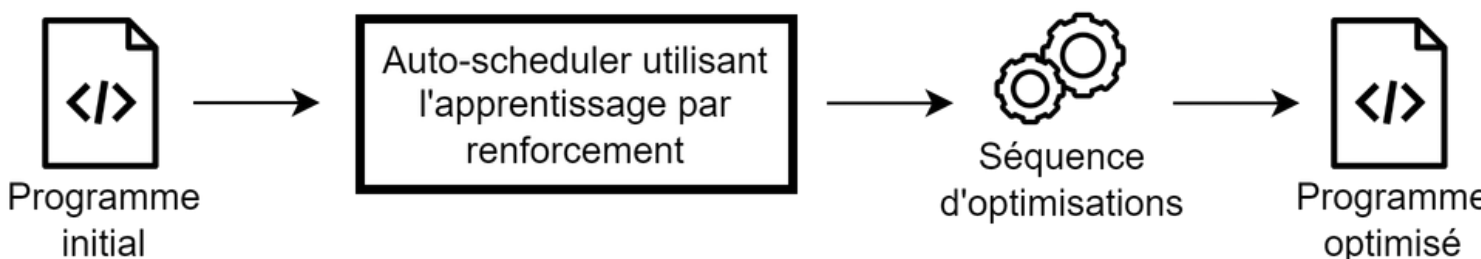
Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



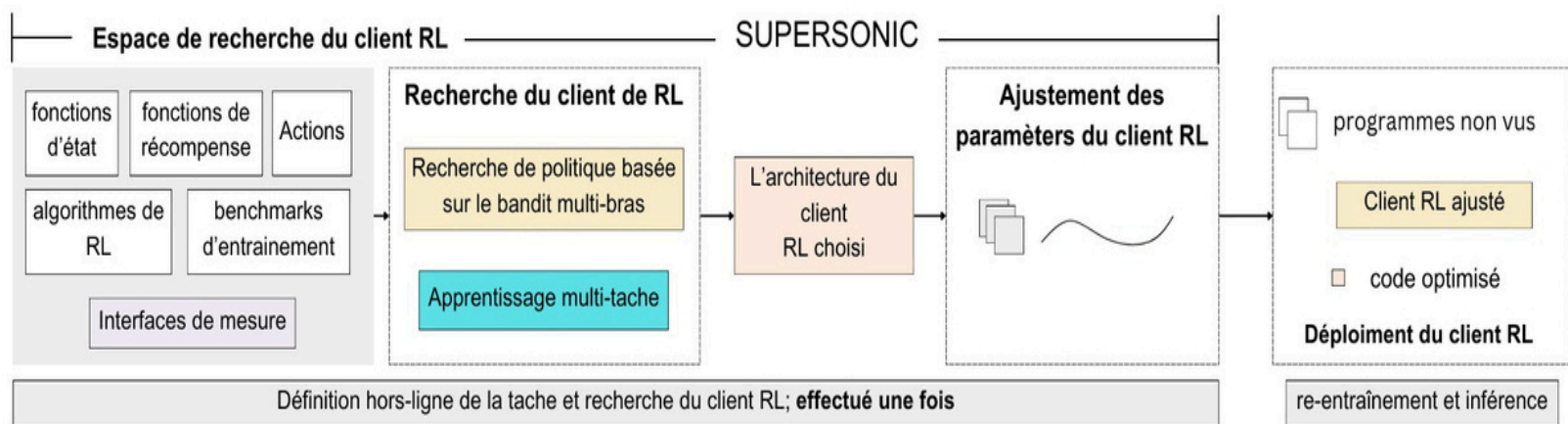
Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



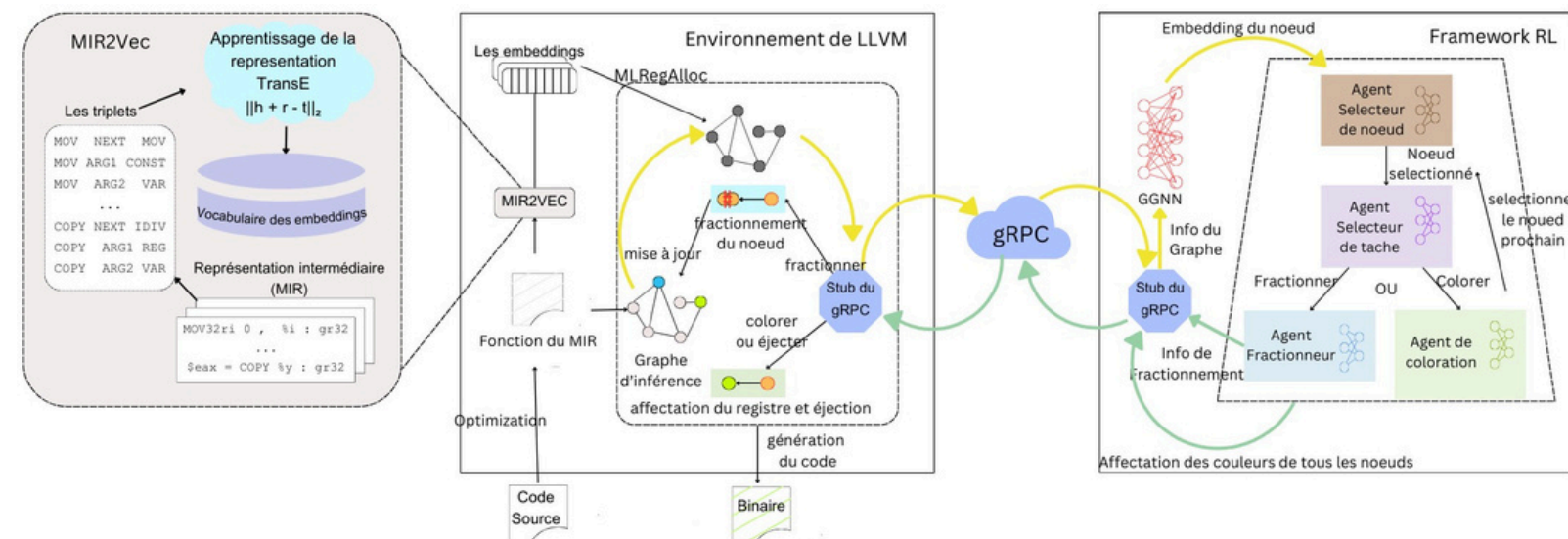
SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement : ajouter des programmes provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

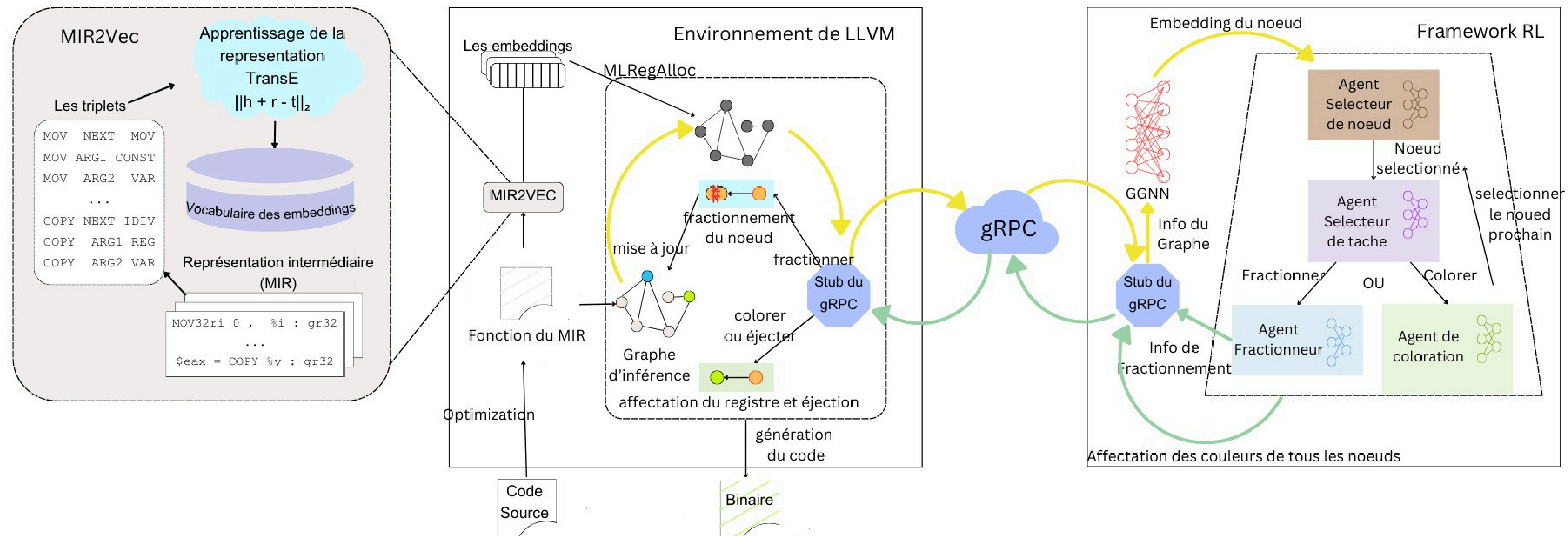
Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas , A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. Acm Sigplan Notices, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAI : Reinforcement Learning for Register Allocation. Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, 129-143.

RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



Encadré par : BAGHDADI Riyadh

9. Limitations et perspectives

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

[illegible]

```

graph LR
    A[Programme initial] --> B[Auto-scheduler utilisant l'apprentissage par renforcement]
    B --> C[Séquence d'optimisations]
    C --> D[Programme optimisé]
  
```

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LIVM	LIVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta- optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

Encadré par : BAGHDADI Riyadh

9. Limitations et perspectives

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

3. Problématique

A blue robot with a square body, round eyes, and a small screen on its chest stands next to a large grey gear. The gear has red arrows indicating a clockwise cycle and contains the code symbols '</>' in green.

Halide [3]

```

graph LR
    A[Pipeline de traitement d'image] --> D[Reinforcement Learning]
    B[Options d'ordonnement] --> D
    D --> C[Ordonnement  
Génération et Estimation  
Raffinement]
    C --> E[Compilation et exécution]
    E -- "Temps d'exécution mesuré" --> D
    D -- "Option d'ordonnement sélectionnée" --> E
  
```

```

graph LR
    A[Programme initial] --> B[Auto-scheduler utilisant l'apprentissage par renforcement]
    B --> C[Séquence d'optimisations]
    C --> D[Programme optimisé]
  
```

[illegible]

8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta- optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAl: Reinforcement Learning for Register Allocation. *Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction*, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*, 129-143.

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement :
ajouter des programmes

provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

Réalisé par : KASMI Abderrahmene & SALMA Samy

Encadré par : BAGHDADI Riyadh

1. Résumé

dans l'intégration de l'apprentissage par renforcement l'optimisation du code permet aux compilateurs d'apprendre et d'ajuster leurs stratégies sans programmation explicite, améliorant ainsi l'efficacité et ouvrant la voie à de nouvelles optimisations. Cette étude explore les techniques classiques d'optimisation dans les compilateurs et les applications actuelles de l'apprentissage par renforcement dans ce domaine.

Mots clés : Apprentissage par renforcement, compilateurs, optimisation automatique du code.

2. Contexte

Ces dernières années, l'augmentation des besoins en calculs complexes a rendu l'optimisation du code essentielle pour améliorer l'efficacité. Les techniques traditionnelles, basées sur des approches manuelles, sont limitées face aux nouvelles architectures. L'apprentissage par renforcement, en revanche, offre une approche adaptable et dynamique pour optimiser automatiquement le code, sans nécessiter de stratégies prédéfinies.

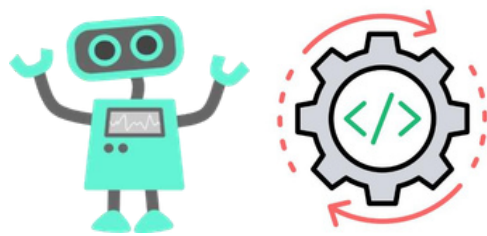
3. Problématique

Représentation des états et des actions : défi de modélisation des caractéristiques du programme pour capturer les informations pertinentes.

Espace d'exploration immense : importance d'intégrer des mécanismes pour explorer intelligemment cet espace, en équilibrant exploration et exploitation.

Évaluation des récompenses : Les récompenses doivent refléter les gains de performance réels.

Comment intégrer efficacement l'apprentissage par renforcement dans les compilateurs pour optimiser le code automatiquement ?



4. Objectifs

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les compilateurs pour améliorer l'optimisation du code.
- Synthétiser les approches et les travaux existants dans la littérature.

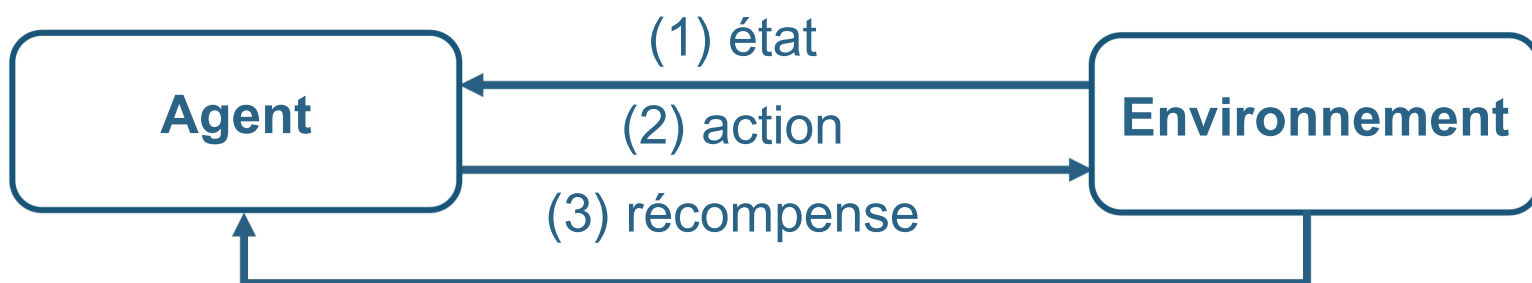
5. Optimisation du code

L'optimisation du code améliore les performances en réduisant le temps d'exécution, la mémoire et la consommation d'énergie. En particulier, les optimisations de boucle telles que le déroulage et la permutation augmentent l'efficacité en minimisant les frais de gestion des boucles et en améliorant la localité des données [2].



6. Apprentissage par renforcement

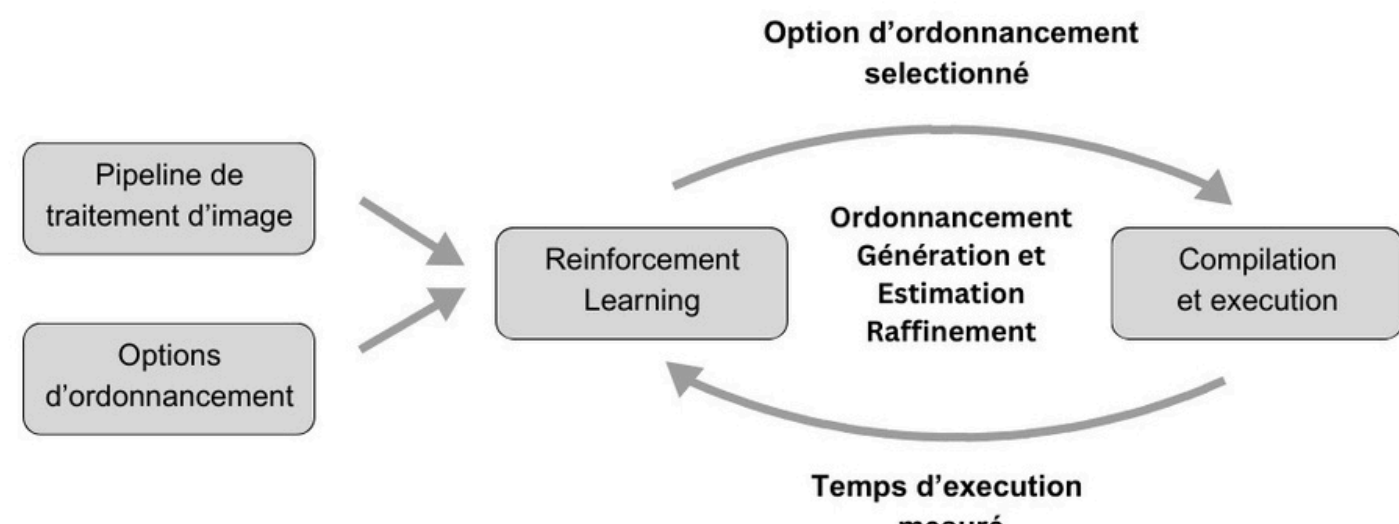
L'apprentissage par renforcement est une méthode d'apprentissage automatique où un agent apprend à prendre des décisions en interagissant avec un environnement et en recevant des récompenses ou des pénalités pour ses actions [4].



7. Apprentissage par renforcement et compilateurs

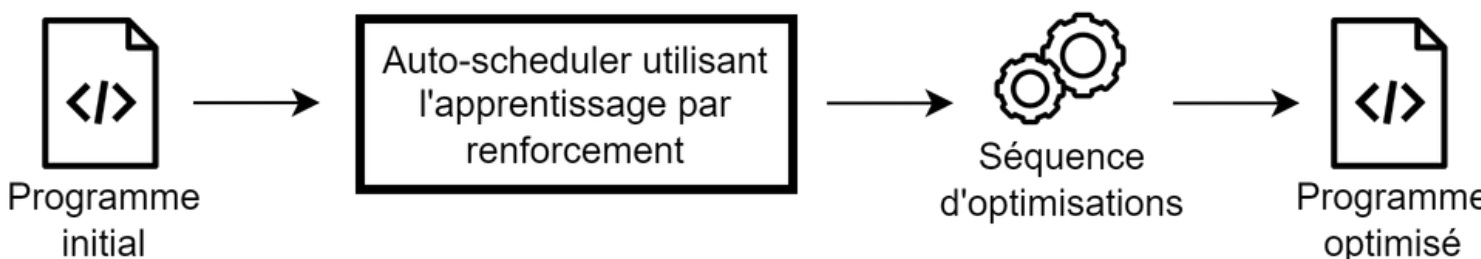
Halide [3]

Un compilateur conçu pour le traitement d'images et de tableaux, optimisant le code pour les architectures hétérogènes en utilisant des pipelines de traitement à plusieurs étapes et en séparant la définition de l'algorithme de l'ordonnancement.



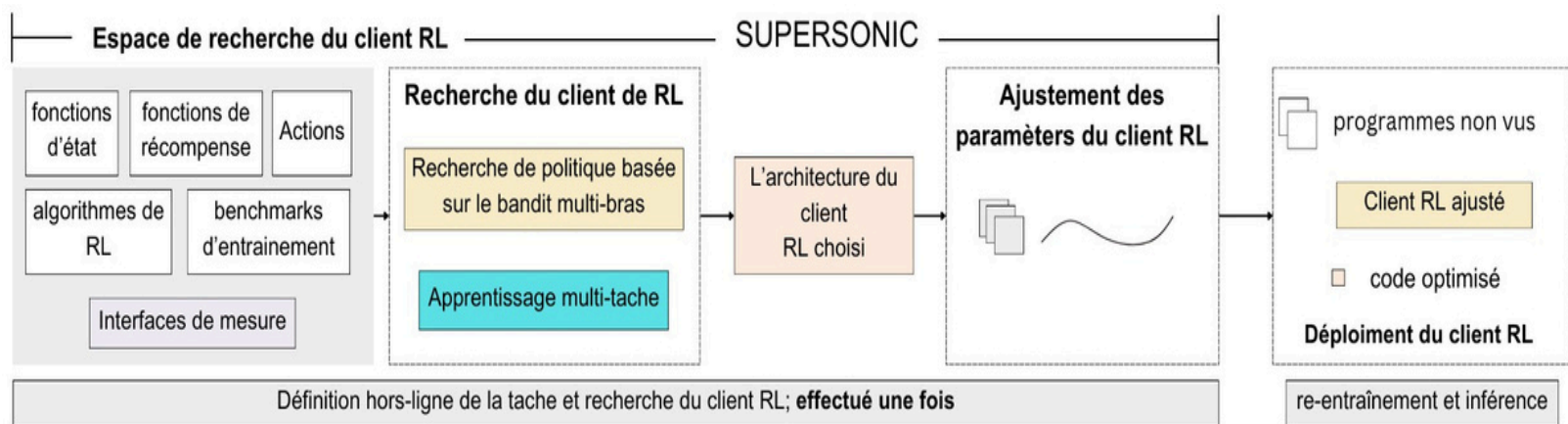
Tiramisu [1]

Un compilateur polyédrique qui génère du code haute performance pour diverses plateformes, en utilisant une représentation intermédiaire à quatre niveaux et un ordonnanceur automatique optimisant les transformations de code.



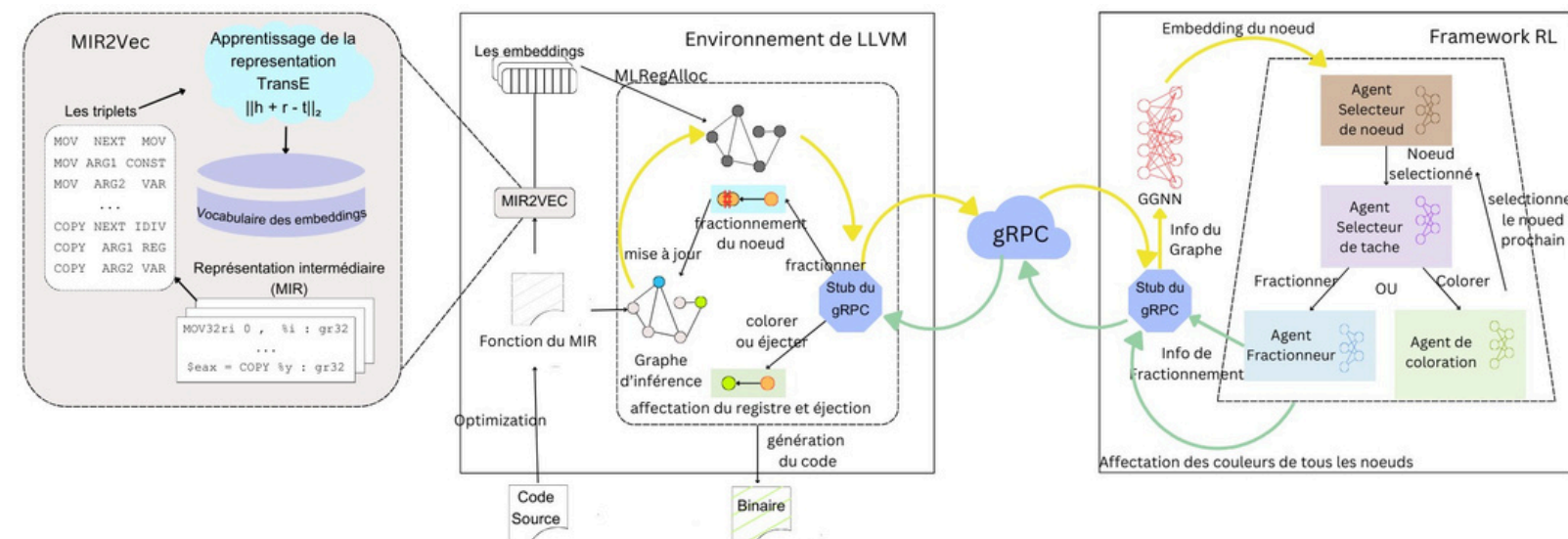
SuperSonic [6]

Un framework open source qui permet d'intégrer l'apprentissage par renforcement dans les compilateurs. Il utilise un méta-optimiseur pour choisir automatiquement l'architecture RL la plus adaptée à un problème donné, en combinant différents algorithmes et fonctions de représentation. SuperSonic s'appuie sur RLib et CompilerGym, et prend en charge de nombreux algorithmes et composants RL.



RL4REAL [5]

Utilise l'apprentissage par renforcement hiérarchique pour optimiser l'allocation de registres. Chaque sous-tâche, comme le fractionnement et l'éjection, est gérée par un agent spécialisé, améliorant ainsi la performance par rapport aux méthodes heuristiques traditionnelles tout en respectant les contraintes strictes de l'allocation de registres.



8. Synthèse

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LLVM	LLVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

9. Limitations et perspectives

Halide

Enrichissement des données d'entraînement : ajouter des programmes provenant d'applications réelles où le système produit des ordonnancements moins performants permet d'améliorer la qualité de l'entraînement.

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

11. Références bibliographiques

[1] Baghdadi , R., Ray , J., Romdhane , M. B., Del Sozzo , E., Akkas , A., Zhang , Y., Suriana , P., Kamil , S., & Amarasinghe , S. (2019). Tiramisu : A polyhedral compiler for expressing fast and portable code. 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 193-205. [2] Kennedy , K., & Allen , J. R. (2001). Optimizing compilers for modern architectures : a dependence-based approach. Morgan Kaufmann Publishers Inc. [3] Ragan-Kelley , J., Barnes , C., Adams , A., Paris , S., Durand , F., & Amarasinghe , S. (2013). Halide : a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. Acm Sigplan Notices, 48(6), 519-530. [4] Sutton , R. S., & Barto , A. G. (2018). Reinforcement learning : An introduction. MIT press. [5] VenkataKeerthy , S., Jain , S., Kundu , A., Aggarwal , R., Cohen , A., & Upadrasta , R. (2023). RL4ReAI : Reinforcement Learning for Register Allocation. Proceedings of the 32nd ACM SIGPLAN International Conference on Compiler Construction, 133-144. [6] Wang , H., Tang , Z., Zhang , C., Zhao , J., Cummins , C., Leather , H., & Wang , Z. (2022). Automating reinforcement learning architecture design for code optimization. Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction, 129-143.

10. Conclusion

Cette étude fournit une analyse approfondie de l'utilisation de l'apprentissage par renforcement pour l'optimisation automatique du code, offrant une alternative flexible aux méthodes traditionnelles. Elle présente d'abord les bases et les techniques d'optimisation telles que la permutation et la parallélisation. Ensuite, elle introduit les concepts de l'apprentissage par renforcement. Et enfin, elle examine des applications concrètes dans divers compilateurs, mettant en évidence les avantages de cette approche tout en soulignant les défis restants.

Encadré par : BAGHDADI Riyadh

9. Limitations et perspectives

Élargissement de l'espace de recherche : l'enrichissement permet d'étendre l'algorithme à de nouveaux domaines et d'intégrer de nouvelles directives. Pour mieux s'adapter aux nouvelles architectures, une approche moins dépendante de l'ingénierie manuelle, tout en exploitant les passes d'analyse de Halide, pourrait être plus efficace.

10. Conclusion

```

graph LR
    A[Programme initial] --> B[Auto-scheduler utilisant l'apprentissage par renforcement]
    B --> C[Séquence d'optimisations]
    C --> D[Programme optimisé]
  
```

	Ordonnanceur de Tiramisu	Ordonnanceur de Halide	SuperSonic	RL4REAL	Neuro Vectoriser	PolyGym	CompilerGym
Compilateur	Tiramisu	Halide	-	LIVM	LIVM	-	Multiple
Problème traité	Optimisation de boucles	Optimisation de pipelines	Optimisation des architectures RL	Allocation de registres	Vectorisation de boucles	Optimisation de boucles	Multiple
Utilise le modèle polyédrique	Oui	Non	Non	Non	Non	Oui	Non
Utilise un modèle de coût	Oui	Non	Non	Non	Non	Non	Non
Type de solution RL	Agent	Agent	Méta-optimiseur	Multi-agent	Agent	Environnement	Environnement
Algorithme	PPO	PPO	Variante de A3C	PPO	PPO	-	-
Nombre d'actions	32	Flexible	Flexible	Dépend du programme	Configurable	Configurable	124
Représentation d'états	Modèle de coût	-	-	MIR2Vec	code2vec	-	-
Récompense	Taux d'accélération	Taux d'accélération	Taux d'accélération / Taille du code	Débit du code	Taux d'accélération	Validité de l'ordonnancement	Taille du code / Taille du binaire / Temps d'exécution

- Analyser les techniques d'optimisation du code.
- Explorer les approches de l'apprentissage par renforcement.
- Illustrer l'intégration de l'apprentissage par renforcement dans les
- compilateurs pour améliorer l'optimisation du code.

Synthétiser les approches et les travaux existants dans la littérature.